Flow Unofficial Documentation

Table of contents

1. Flow Unofficial Documentation 3
1.1 Terms Used in This Documentation 4
1.2 Features 4
1.3 Limitations 5
1.4 Security 5
1.5 Other Documentation 5
2. Getting Started 6
2.1 Basic Example 6
2.2 Updating Forms 7
2.3 Context and Redraws at a Glance 12
3. Elements/widgets 14
3.1 Common Aspects 14
3.2 Layout Elements 23
3.3 Functional Elements 29
3.4 Hypertext Markup 62
4. Other features 66
4.1 Unsupported Features 66
4.2 Security 71
4.3 Styling Forms 73
4.4 Padding, Spacing, and Backgrounds 85
4.5 Tooltips 87
4.6 Hiding Elements 89
4.7 Using a Form as an Inventory 91
4.8 Using a Form as a HUD 93
4.9 Manual Positioning 97
4.10 Experimental Features 101
4.11 Popovers (Highly Experimental) 111

1. Flow Unofficial Documentation

Flow is an experimental layout manager and formspec API replacement for Luanti (formerly Minetest). Describe your menu, and flow handles the rest. Vaguely inspired by Flutter and GTK.

Luanti's built-in menu format (inventories, chests, dialogs, ...) is *formspec*, consisting of one long string of buttons, text fields, and inventory slots. With *formspec*, you have to manually place every element by position and size, consult official docs in case of the many gotchas, and there's no reasonable way to write an LSP for it.

Flow lets you describe the same menu with plain Lua tables, positions and sizes everything for you, and generates the formspec behind the scenes. You never have to write one. And if you like using modern IDE features, you will now actually have direct access to definitions **from the editor** (LuaCATS annotations are included).

Online tutorial/demo (source)

1.1 Terms Used in This Documentation

Term	Meaning
Form	A menu made with flow, created with `flow.make_gui()` and shown with `form:show()`. This documentation does not call it a "gui".
Formspec	Luanti's own text format for menus. Flow generates formspecs for you.
Element	One part of a form, such as a label, a button, or a box. Elements are made with the functions in `flow.widgets` (the name of that table is the only place "widget" is used).
Build function	The function you pass to `flow.make_gui()`. It returns the elements of the form and is called every time the form is drawn.
Root element	The element that the build function returns, which holds everything else in the form.
Container	An element that holds other elements. `gui.Container` is the one whose elements you place yourself with `x` and `y`.
Layout element	An element that arranges the elements inside it, such as `gui.VBox`, `gui.HBox`, and `gui.Stack`, see Layout Elements.
Flow	The mod. `gui.Flow` (written with the `gui.` prefix) is an element of the mod that wraps its children onto new rows.
`ctx`	The context table of a form, see Getting Started.

1.2 Features

1.2.1 Layouting

- **Automatic placement.** Flow works out every position, so there's no arithmetic to redo when you add one button.
- **Comprehensive layout elements.** Nest `HBox` and `VBox` containers and they line up on their own. Scrolling, paging, stacking, wrapping, and flexible spacing are covered too. And yes, also manual placement using `Container`.
- **Automatic sizing.** Labels, checkboxes, buttons, text fields, password fields, text areas, drop-downs, text lists, and vertical labels size themselves. See Sizes.
- **Room to grow.** Set `expand = true` and an element fills any spare space.
- **Translation-friendly.** Elements sized from their label grow or shrink to fit longer text[1].

1.2.2 Other Features

- **No form names.** They still exist internally, but you never have to deal with them.
- **No state bookkeeping.** Flow remembers the values of fields, scrollbars, checkboxes, and the like across redraws, and reapplies them for you.
- **Built-in debugging.** The inspector mod helps you develop and debug forms.
- **Safer input handling.** Flow mitigates some common formspec input security issues. See Security.

1.3 Limitations

- This mod doesn't support all of the features that regular formspecs do. See Unsupported Features for a list.

1.4 Security

Flow ignores potentially malicious formspec input from clients, such as buttons or fields that haven't been shown to the client, out-of-bounds dropdown selections, and newlines in `Field` elements (where it's impossible to enter a newline without pasting it in). See Security for the full list of what flow checks and what you still need to check yourself.

1.5 Other Documentation

The official documentation is available at https://luk3yx.gitlab.io/minetest-flow/. Some code snippets have a "run" button which will open them in a web-based playground, not all of these will work properly as the playground doesn't support all formspec elements.

---

1. This is an estimate, not a measurement. The real size of a label depends on the client's font, DPI, window size, and formspec scaling settings, and Luanti gives mods no way to measure text. Flow assumes 0.21 slots per character (UTF-8 characters, not bytes, counting the longest line) and 0.4 slots per line of height, scaled by any `*1.5`-style `font_size` style. It ignores the font. If the estimate is off for your text, set `w`, `h`, or `min_w` yourself. ↩

2. Getting Started

A *form* is a menu that is shown to a player. It is made of *elements* (also called widgets), such as labels, buttons, text fields, and inventory slots, which are arranged by layout elements like `gui.VBox`. You describe a form with a *build function*, which is the function you pass to `flow.make_gui()`. It returns the elements, and flow works out the positions and sizes and shows the form for you. This documentation always calls it the build function. See Functional Elements for what each element does.

2.1 Basic Example

See `example.lua` in the flow repository for a more comprehensive example which demonstrates how layouting and alignment works. In singleplayer, a player with the `server` privilege can also run `/flow-example` to open it in the game.

The build function is called with the player and a context table (`ctx`) every time the form is drawn. `ctx` belongs to one player's copy of the form: flow keeps the same table for as long as the form is open, passes it to the build function and to every `on_event` function, and you can store anything in it. Flow adds a `ctx.form` table to it, which holds the current values of the form's named inputs (see Names and Values).

Three things in the example below look alike but are different:

Name in the example	What it is	Who writes it
`ctx.value`	The starting `ctx`: whatever you passed to `form:show`.	You, before the form is shown.
`ctx.form.cool_dropdown`	The live value of the input named `cool_dropdown`.	Flow: it sets the initial value on the first draw and updates it when the player changes the input.
`ctx.picked`	Your own state.	Your `on_event` function, when the player presses the button.

All three are in the same table. Do not use `form` as a key of your own and do not replace `ctx.form`, flow raises an error if you do that in the build function.

`form:show(player, ctx)` shows the form to a player. The second argument is the starting `ctx`. Flow uses that table itself instead of a copy, and uses a new empty table if you leave it out.

The example below uses a player called `playground`, which only exists in the online playground. In a mod, use a real player object instead, such as the one passed to a chat command or to `core.register_on_joinplayer`.

```lua
local gui = flow.widgets -- local recommended for direct referencing
local STARTING_SELECTION = 3 -- change and rerun to see the difference

local function build_vbox(player_obj, ctx)
    return gui.VBox {
        gui.Label { label = "Here is a dropdown:" },
        gui.Dropdown {
            name = "cool_dropdown",
            items = { 'First item', 'Second item', 'Third item' },
            index_event = true,
            selected_idx = ctx.value or 1
        },
        gui.Button {
            label = "Get dropdown index",
            on_event = function(player, ctx)
                local idx = ctx.form.cool_dropdown or ctx.value or 1
                core.chat_send_player(player:get_player_name(),
                    ("You have selected item #%d!"):format(idx))

                -- Remember the index and redraw so the label below shows it.
                ctx.picked = idx
                return true
            end,
        },
        gui.Label {
            label = ctx.picked
                and ("You have selected item #%d!"):format(ctx.picked)
                or "Pick an item, then press the button.",
        },
    }
end

local form = flow.make_gui(build_vbox)
-- "singleplayer" is the default player name for single-player games
local player_obj = core.get_player_by_name("singleplayer")

form:show(player_obj, { value = STARTING_SELECTION }) -- Starting ctx, optional
```

Pick an item and press the button: the button's `on_event` function stores the index in `ctx.picked` and returns `true`, which redraws the form so that the label under the button changes. The same message is also sent to the chat.

2.2 Updating Forms

If some data displayed inside a form changes (for example a timer or progress indicator), you can use `my_gui:update(player)` or `my_gui:update_where(func)` (see below) to update the form without resetting `ctx` or showing the form again if the player has closed it.

Luanti has no way to change a form that is already open, so flow has to show the whole form again. Because of this, players may lose text typed into fields that hasn't been sent to the server (which happens when they press a button or Enter) when the form is updated.

Here is a form with one button for each way to update it. The counter is a plain variable, not something stored in `ctx`, so flow cannot notice when it changes. Each button adds 1 to it and then redraws the forms that should show the new number.

2.2.1 Example

```lua
local gui = flow.widgets
local counter = 0 -- External state; flow cannot track its changes.
local my_gui

local function add_button(label, update)
    return gui.Button {
        label = label,
        on_event = function(player, ctx)
            counter = counter + 1
            update(player, ctx)
        end,
    }
end

my_gui = flow.make_gui(function(player, ctx)
    ctx.test = 123 -- Used by update_where() below.
    return gui.VBox {
        gui.Label { label = "Counter: " .. counter },
        add_button("Add 1 and update my form", function(p)
            my_gui:update(p)
        end),
        add_button("Add 1 and update forms with ctx.test == 123", function()
            my_gui:update_where(function(_, c) return c.test == 123 end)
        end),
        add_button("Add 1 and update forms of admins", function()
            my_gui:update_where(function(p)
                return core.check_player_privs(p:get_player_name(), "server")
            end)
        end),
        add_button("Add 1 and update everyone's form", function()
            my_gui:update_where(function() return true end)
        end),
    }
end)
```

Inside an `on_event` handler, you can use `return true` instead of `my_gui:update(player)`. This form does the same as the first button above:

```
local counter = 0

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = "Counter: " .. counter },

        gui.Button {
            label = "Add 1 and update the form",
            on_event = function(player, ctx)
                counter = counter + 1
                return true -- Same as my_gui:update(player)
            end,
        },
    }
end)
```

If you're using a form as a HUD, you must call `my_gui:show_hud(player)` to update it. See HUDs for how to show and close one.

Here is a complete example you can run. A timer redraws the form every second with `update_where`, and the click counter stored in `ctx` survives every redraw because updating a form does not reset `ctx`:

```
local gui = flow.widgets

-- Shared by every open copy of the form.
local seconds = 0

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = ("Form drawn after %d seconds"):format(seconds) },
        gui.Label { label = ("Clicks (kept in ctx): %d"):format(ctx.clicks or 0) },
        gui.Button {
            label = "Click me",
            on_event = function(player, ctx)
                ctx.clicks = (ctx.clicks or 0) + 1
                return true -- redraw this form
            end,
        },
    }
end)

-- Redraw every open copy once per second.
local function tick()
    seconds = seconds + 1
    form:update_where(function() return true end)
    core.after(1, tick)
end
core.after(1, tick)

form:show(core.get_player_by_name("current_player"))
```

2.2.2 Redrawing When a Value Changes

You often do not need `return true`. If your build function reads the value of a named input from `ctx.form`, flow notices and redraws the form by itself whenever the player changes that value.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Checkbox { name = "advanced", label = "Show advanced options", selected = true },

        -- Reading ctx.form.advanced here makes flow redraw the form
        -- whenever the checkbox changes.
        ctx.form.advanced and gui.Label { label = "Advanced options" }
            or gui.Nil {},
    }
end)
```

The rules are:

- Only reads while the build function runs count, including reads in helper functions that it calls. Reading `ctx.form` inside an `on_event` function does not.
- The form is only redrawn if the value is different from the stored one.
- If the element has its own `on_event` function, there is no automatic redraw. That function has to `return true` instead.
- Text fields only send their text when the player presses a button or Enter (see Names and Values), so the redraw happens then, and only if the text is different from the stored one. Pressing a button with the text unchanged does not redraw because of the field.

Here are the two cases that surprise people. The label shows the values that the build function reads, but only `Show details` redraws the form:

```
local gui = flow.widgets

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- No on_event, and the build function reads it below: ticking this
        -- redraws the form.
        gui.Checkbox { name = "details", label = "Show details" },

        -- Has its own on_event: ticking this does NOT redraw the form, so
        -- the label below goes out of date. Return true to fix that.
        gui.Checkbox {
            name = "loud", label = "Loud",
            on_event = function(player, ctx) end,
        },

        gui.Label {
            label = ("details = %s, loud = %s"):format(
                tostring(ctx.form.details), tostring(ctx.form.loud)),
        },

        -- Reading ctx.form.details here does not make the form redraw when
        -- details changes. Only the reads above (in the build function) do.
        gui.Button {
            label = "Report",
            on_event = function(player, ctx)
                core.chat_send_player(player:get_player_name(),
                    "details is " .. tostring(ctx.form.details))
            end,
        },
    }
end)
```

2.3 Context and Redraws at a Glance

2.3.1 Context Lifetime by Mode

How long `ctx` lives depends on how the form is shown:

How the form is shown	Where `ctx` comes from	How long it is kept
`my_gui:show(player, ctx)`	The table you pass, or a new empty one.	Until the form is closed. `update`, `update_where` and `return true` keep it. Calling `show` again starts with a new one.
`my_gui:set_as_inventory_for(player, ctx)`, see Inventory	The table you pass. If you pass none and this form is already the player's inventory, the existing `ctx` is reused, otherwise a new empty one.	Until the form is unset or the player leaves. Passing a new `ctx` replaces the old one.
`my_gui:show_hud(player, ctx)`, see HUDs	The table you pass, or a new empty one every time.	Not kept: flow remembers nothing between calls, so pass your own table each time.
`my_gui:render_to_formspec_string(player, ctx)`, see Rendering to a Formspec	The table you pass, or a new empty one.	Flow keeps nothing: pass the same `ctx` each time you redraw.

2.3.2 Ways to Redraw a Form

Way	Redraws	Keeps `ctx`	Works for
`return true` from `on_event` or `on_key_enter`	The form of the player who used it	Yes	Forms shown with `show` and inventory forms
Reading `ctx.form.name` in the build function (see Redrawing When a Value Changes)	The form of the player who changed the value	Yes	Forms shown with `show` and inventory forms
`my_gui:update(player)`	One player, if they have the form open	Yes	Forms shown with `show` only
`my_gui:update_where(func)`	Every player with the form open for whom `func(player, ctx)` returns `true`	Yes	Forms shown with `show` only
`my_gui:set_as_inventory_for(player, ctx)`	One player	Reused if you pass none	Inventory forms only
`my_gui:show_hud(player, ctx)`	One player	No	HUDs only
`my_gui:show(player, ctx)`	One player (and the old form is replaced)	No, starts from the `ctx` you pass	Forms shown with `show`

3. Elements/widgets

3.1 Common Aspects

All flow elements (widgets) draw something, change size/position of other elements, or accept input. Every element is a function in `flow.widgets` that takes a table of parameters. For example:

```
local gui = flow.widgets

gui.Label { label = "Hello world!" }
```

3.1.1 Sizes

Sizes are measured in inventory slots: `1` is the width and height of one item slot. There is no hidden padding or spacing, so two boxes that are `1` wide and placed next to each other with `spacing = 0` touch. You never give an element a position, boxes place their children and flow works out the total size of the form.

Elements that show text, such as labels, buttons, and fields, are sized to fit their text unless you give them a `w` and `h`. Elements that show something without a natural size, such as images, boxes, models, and tables, need an explicit `w` and `h` (their parameter lists below do not say `-- Optional`). Flow can only estimate the width of text because the real width depends on the player's font, screen size, and scaling settings, so avoid layouts that rely on exact text widths (see the note on label sizes).

Which Elements Take `w`, `h`, `min_w`, and `min_h`

Elements that size themselves are the ones flow can work out a size for: from their text (labels, buttons, fields, dropdowns, text lists), or from the elements inside them (containers and boxes). For these, `w` and `h` are optional, and `min_w` and `min_h` set a lower limit on the size flow works out. `min_w` and `min_h` only apply to a dimension

that flow works out itself: if you set both `w` and `h`, both minimums are ignored, and if you set only `w`, `min_w` is ignored and `min_h` still applies. Elements that cannot size themselves ignore `min_w` and `min_h`.

Element	`w` and `h`	`min_w` and `min_h`
`gui.Label`	Optional	Yes
`gui.Vertlabel`, `gui.Checkbox`	Not supported, always sized automatically	Yes
`gui.Button`, `gui.ButtonExit`, `gui.ButtonURL`	Optional	Yes
`gui.ImageButton`, `gui.ImageButtonExit`, `gui.ItemImageButton`	Optional, the size is worked out from the label like a button (at least 1.6 by 0.8), so you usually want to set them	Yes
`gui.ButtonUrlExit`	Required, it has no automatic size (see the note below the table)	No
`gui.Field`, `gui.Pwdfield`, `gui.Textarea`, `gui.Dropdown`, `gui.Textlist`	Optional	Yes
`gui.Container`, `gui.ScrollContainer`	Optional, the size is worked out from the elements inside	Yes
`gui.HBox`, `gui.VBox`, `gui.Stack`	Not supported, use `min_w` and `min_h`	Yes
`gui.ScrollableVBox`, `gui.PaginatedVBox`	Optional	Not checked
`gui.Spacer`	Optional	Not checked
`gui.Flow`	`w` required, `h` not supported	No
`gui.Image`, `gui.AnimatedImage`, `gui.ItemImage`, `gui.Model`	Required	No
`gui.Box`, `gui.Table`, `gui.Hypertext`, `gui.Scrollbar`, `gui.Tabheader`	Required	No
`gui.List`	Required, in slots rather than in size units	No

Element	`w` and `h`	`min_w` and `min_h`
`gui.Background`, `gui.Background9`	Required, unless `auto_clip` is `true`	No

An element without a size that needs one is written to the form with an empty size, which Luanti rejects, so you will see it as a broken form rather than an error message. `gui.ButtonUrlExit` has no automatic size, unlike `gui.ButtonURL`, so its `w` and `h` are required, see `gui.ButtonUrlExit`.

3.1.2 Names and Values

Many elements take a `name`. Input elements (fields, checkboxes, dropdowns, lists of text, tables, ...) with a `name` have their current value stored in `ctx.form[name]` and flow remembers it when the form is redrawn:

Element	Value in `ctx.form[name]`
`gui.Field`, `gui.Pwdfield`, `gui.Textarea`	Text (string)
`gui.Checkbox`	`true` or `false`
`gui.Dropdown`	Text of the selected item, or its number if `index_event = true`
`gui.Textlist`, `gui.Table`	Number of the selected row (first row is `1`)
`gui.Tabheader`	Number of the selected tab (first tab is `1`)
`gui.Scrollbar`	Position of the slider (number)
`gui.Hypertext`	`"action:"` and the name of the last clicked `<action>` link

Until the player changes something, the value is the element's initial value (`default`, `selected`, `selected_idx`, `value`, `current_tab`). If you did not give one, it is `""` for text, `false` for checkboxes, `1` for tables, text lists, and dropdowns with `index_event = true`, the text of the first item for other dropdowns, and `0` for scrollbars. `gui.Hypertext` has no starting value, and neither has `gui.Tabheader` without a `current_tab`, so `ctx.form[name]` is `nil` until the player clicks a link or a tab. Text fields only send their text to the server when the player presses a button or Enter, so `ctx.form` can be out of date until then.

If your build function reads `ctx.form.some_name`, flow redraws the form automatically whenever the player changes that value (unless the element has its own `on_event`), see Redrawing When a Value Changes. Buttons do not store a value.

3.1.3 Events

Elements that the player can use accept an `on_event` function, which is called with the player and `ctx`. Return `true` from it to redraw the form. A `name` is optional if you use `on_event`, flow makes one up.

Flow has four callbacks. All of them are called with the player and `ctx`:

Callback	Put it on	Called when	Return `true`
`on_event`	Any element the player can use	The client sends the element's name: a button when it is clicked, an input when the player changes it. A text field is sent with every submission, so its `on_event` is called whenever the player presses any button or Enter.	Redraws the form.
`on_key_enter`, see Handling Enter Keypresses in Fields	`gui.Field`, `gui.Pwdfield`	The player presses Enter in the field. The `on_event` of the same field is called as well.	Redraws the form.
`on_quit`, see Running Code When a Form Is Closed	`gui.Container`, `gui.HBox`, `gui.VBox`, `gui.Stack`	The client closes the form.	Return nothing.
`on_close_popover`, see Popovers	The container that has a `popover`	The player clicks outside the popover.	Redraws the form.

Only one button callback runs for each submission, and it runs after all other callbacks, see Security. This includes `on_close_popover`, which flow handles as a button. `on_event` on a text field is easy to mix up with `on_key_enter`: `on_event` is called on any submission, `on_key_enter` only when Enter is pressed.

3.1.4 Colors

Colors are strings. `"#RGB"` and `"#RRGGBB"` are fully opaque, while `"#RGBA"` and `"#RRGGBBAA"` end with an alpha value, where `00` is fully transparent and `FF` is fully opaque. Color names also work (all the names from CSS, such as `"red"` and `"darkgreen"`), and you can add an alpha value after a `#` to a color name. The alpha value is one or

two hexadecimal digits, so `"darkgreen#08"` is dark green with an alpha of `08` (hex, about 3% opaque) and `"darkgreen#80"` is about half transparent.

Warning: `gui.Box`, and the `bgcolor` of containers (which flow draws with a box), are the exception to the opaque rule: a `"#RGB"` or `"#RRGGBB"` color there is drawn semitransparent. Color names are not affected. See `gui.Box`.

Most text can also be coloured by putting color escape sequences in it, such as the ones made by `core.colorize()`. Dropdown items and tab captions are an exception, they cannot be coloured.

3.1.5 Textures and Items

A texture name is the file name of an image from the `textures` folder of any loaded mod, such as `"default_dirt.png"`. An item name is the registered name of an item or node, such as `"unifieddyes:colorium"`.

3.1.6 Invisible Elements

Some elements draw nothing and take up no space in a `gui.HBox`, `gui.VBox`, or `gui.Stack`. They change how a nearby element looks or behaves, and where you put them matters:

Element	Where it goes	What it affects
`gui.Style`	Before	The element with that `name`
`gui.StyleType`	Before	Every element of that type after it
`gui.SetFocus`	Before	The element with that `name`
`gui.Listcolors`	Before	The `gui.List` elements after it
`gui.TableColumns`	Before	The `gui.Table` after it
`gui.TableOptions`	Before	The `gui.Table` after it
`gui.ScrollbarOptions`	Before	The `gui.Scrollbar` elements after it
`gui.Tooltip`	After	The element with the `name` in `gui_element_name`
`gui.Listring`	After	The `gui.List` elements it names, or the last two lists before it if it has no parameters

The `style` and `tooltip` parameters that every element has are the exception: flow inserts the matching style or tooltip element directly before the element for you, so you do not need to place anything, see Styling and Tooltips. The "before" rule for the others comes from Luanti, and so does the "after" rule of `gui.Tooltip`. An invisible

element is skipped when flow works out sizes and spacing, so putting one between two elements does not add a gap. `gui.Nil` is one as well: it is a tooltip that is hidden, which is why it takes no space.

3.1.7 Parameters Shared by All Elements

Every element, including Layout Elements, accepts these parameters in addition to its own:

Parameter	Description
`padding`	Space around the element, see Padding.
`visible`	Set to `false` to hide the element but keep its space, see Hiding Elements.
`expand`	Set to `true` to make the element grow to fill spare space in the `gui.HBox` or `gui.VBox` it is in. If several elements expand, they share the spare space equally.
`align_h`, `align_v`	Where the element goes when there is more room than it needs: `"start"`, `"end"`, `"center"`, `"fill"` (stretch), or `"auto"` (the default, flow picks one). `"left"` and `"top"` mean `"start"`, `"right"` and `"bottom"` mean `"end"`, and `"center"` means `"center"`.
`min_w`, `min_h`	The smallest size of the element, for elements that size themselves, see Which Elements Take `w`, `h`, `min_w`, and `min_h`.
`tooltip`	Text to show when the player hovers over the element, see Tooltips.
`style`	Inline styles, see Styling.
`on_event`	A function to call when the player uses the element, see Events above.

Automatic Alignment

`"auto"` only matters when an element is given more room than it needs. That happens across a box (the width in a `gui.VBox`, the height in a `gui.HBox`), along a box for elements with `expand = true`, and for every element in a `gui.Stack`. Flow then picks by the type of the element:

Element	What `"auto"` does
`gui.Image`, `gui.AnimatedImage`, `gui.Model`, `gui.ItemImageButton`	Centered.
`gui.List`	Centered (a list cannot be stretched).
`gui.Label` (vertically)	Centered in the spare height.
`gui.Label` (horizontally)	Stays at the start, unless it has `expand = true` and the spare room is along the box. It is then stretched, which centers the text (see the centering hack in `gui.Spacer`).
`gui.Checkbox`	Centered vertically. Horizontally it stays at the start, unless it has `expand = true` and the spare room is along the box, then it is centered.
Everything else (buttons, fields, boxes, containers, ...)	Stretched to fill the spare room.

Set `align_h` or `align_v` yourself to override this. Which elements need a `w` and `h`, and which do not, is a separate question, see Sizes above.

3.1.8 Nine-Slice Images

Images and image-like backgrounds can optionally be drawn as a "nine-slice". The picture is cut into a 3x3 grid, where the corners keep their size, the edges are stretched in one direction, and the middle is stretched in both. This lets one small texture make a panel or border of any size without distorting its corners. The `middle_*` parameters describe where the cuts are, in texture pixels (whole numbers):

- `middle_x` only: the cuts are `middle_x` pixels from every side.
- `middle_x` and `middle_y`: `middle_x` pixels from the left and right sides and `middle_y` pixels from the top and bottom.
- All four: the middle rectangle starts at `middle_x, middle_y` and ends at `middle_x2, middle_y2`. Negative `middle_x2` and `middle_y2` values are measured from the far (right/bottom) edge of the texture.

Here is the grid. The two vertical cuts are at `x` and `x2`, and the two horizontal cuts are at `y` and `y2` (the names are those of the `middle_*` parameters):

```
     x               x2
     |               |
  +---+---------------+---+
  | 1 |       2       | 3 |  <- above y
y-+---+---------------+---+
  | 4 |       5       | 6 |
  |   |               |   |
  +---+---------------+---+
y2| 7 |       8       | 9 |  <- below y2
  +---+---------------+---+
```

Parts 1, 3, 7, and 9 (the corners) keep their size. Parts 2 and 8 are stretched sideways, parts 4 and 6 are stretched up and down, and part 5 is stretched in both directions.

Nine-Slice Formats

The same cuts are written in three ways, depending on where they are used:

Where	Parameter	How it is written	Meaning
Elements with `middle_*` parameters, such as `gui.Background9` and `gui.AnimatedImage`	`middle_x`, `middle_y`, `middle_x2`, `middle_y2`	Separate numbers	As described above.
`bgimg_middle` on the root element or a box or container, see Padding	`bgimg_middle`	One number	Same as `middle_x` alone: that many pixels from every side.
`bgimg_middle` and `fgimg_middle` in a button style	`bgimg_middle`, `fgimg_middle`	A string: `"x"`, `"x,y"`, or `"x,y,x2,y2"`	The same as `middle_x`; `middle_x` and `middle_y`; or all four.

3.2 Layout Elements

You should do `local gui = flow.widgets` in your code to improve readability. All examples will assume that this line exists.

These elements control how other elements are arranged. Most of them draw nothing themselves: they place the elements inside them, work out how big everything is, and share out any spare space. Elements that draw something or accept input (labels, buttons, fields, ...) are described in Functional Elements. To place elements at exact coordinates instead, see Manual Positioning.

Warning: `gui.VBox`, `gui.HBox`, and `gui.Stack` do not support `w` and `h` properties and will break in weird ways if you try and use them. Use `min_w` and `min_h` instead. Unlike images or boxes, which need a `w` and `h` because they have no content to measure, these elements always work out their size from the elements inside them. If you set both, flow skips placing those elements, so the children end up without positions. The table of which elements take `w` and `h` is in Common Aspects.

The Layout elements that do take `w` and `h` are `gui.ScrollableVBox` and `gui.PaginatedVBox` (both are optional), and `gui.Spacer`. `gui.Flow` needs a `w` but does not support `h`, see Experimental.

3.2.1 `gui.VBox`

A vertical box, similar to a VBox in GTK. Elements inside a VBox are stacked vertically.

```
gui.VBox {
    -- These elements are documented later on.
    gui.Label { label = "I am a label!" },

    -- The second label will be positioned underneath the first one.
    gui.Label { label = "I am a second label!" },
}
```

Elements inside boxes have a spacing of 0.2 between them. To change this, you can add `spacing = <number>` to the box definition. For example, `spacing = 0` will remove all spacing between the elements. `gui.HBox` and `gui.Flow` have the same `spacing` parameter and default. Spacing is the gap between the elements of a box, while padding is the space around a single element.

3.2.2 `gui.HBox`

Like `gui.VBox` but stacks elements horizontally instead.

```
gui.HBox {
    -- These elements are documented later on.
    gui.Label { label = "I am a label!" },

    -- The second label will be positioned to the right of first one.
    gui.Label { label = "I am a second label!" },

    -- You can nest HBox and VBox elements
    gui.VBox {
        gui.Image { w = 1, h = 1, texture_name = "default_dirt.png", align_h = "center" },
        gui.Label { label = "Dirt", expand = true, align_h = "center" },
    }
}
```

3.2.3 `gui.ScrollableVBox`

Similar to `gui.VBox`, but shows its contents in an area of a fixed size that the player can scroll, and automatically adds a scrollbar next to it. Anything outside the area is hidden until it is scrolled into view. The area is `w` by `h`, both are optional: the width defaults to the width of the contents and the height defaults to the height of the contents, up to a maximum of 5.

On clients too old to support scrolling areas (before Luanti 5.4), flow shows one page of the contents at a time instead, with `<` and `>` buttons to change page.

To change how the scrollbar looks, set `custom_scrollbar` to a table of parameters for `gui.Scrollbar`, for example `custom_scrollbar = {w = 0.8}` to make it wider.

```
gui.ScrollableVBox {
    -- A name must be provided for ScrollableVBox elements. You don't
    -- have to use this name anywhere else, it just makes sure flow
    -- doesn't mix up scrollbar states if one gets removed or if the
    -- order changes.
    name = "vbox1",

    -- Specifying a height is optional but is probably a good idea.
    -- If you don't specify a height, it will default to
    -- min(height_of_content, 5).
    -- This is shorter than the contents, so a scrollbar is added.
    h = 2,

    -- These elements are documented later on.
    gui.Label { label = "I am a label!" },

    -- The second label will be positioned underneath the first one.
    gui.Label { label = "I am a second label!" },

    -- The rest is hidden until the player scrolls down.
    gui.Label { label = "I am a third label!" },
    gui.Label { label = "I am a fourth label!" },
    gui.Label { label = "I am a fifth label!" },
}
```

3.2.4 `gui.Spacer`

A "flexible space" element that expands by default. Example usage. A box is only as wide as its contents unless you make it wider, and a spacer can only fill space that is left over, so this box has a `min_w` and a background color to show how much room there is:

```
gui.HBox {
    -- Without this, the box would be exactly as wide as the buttons and
    -- there would be no space left for the spacer to fill.
    min_w = 12,

    -- Only here to show the edges of the box.
    bgcolor = "#335577",

    -- These buttons will be on the left-hand side of the box
    gui.Button { label = "Cancel" },
    gui.Button { label = "< Back" },

    gui.Spacer {},

    -- These buttons will be on the right-hand side of the box
    gui.Button { label = "Next >" },
    gui.Button { label = "Confirm" },
}
```

Prefer `expand = true` with `align_h` or `align_v` when that gives the same result. Spacers are implemented as hidden containers, so a few things behave differently from other elements:

- A spacer counts as an element when flow adds the `spacing` between the elements of a box. Each spacer therefore adds a gap on each side of it, which is taken out of the spare room it fills. If the box has no spare room, the spacer has no width but the gaps are still there.
- A spacer cannot have child elements and flow raises an error if you give it any.
- Several spacers (or other elements with `expand = true`) in the same box share the spare space equally.

What `"auto"` alignment (the default for `align_h` and `align_v`) picks for each kind of element is described in Automatic Alignment. For a spacer it means "fill", so the spacer takes all of the spare space it is given.

You can get the same layout without a spacer, however the code doesn't look as clean. Flow puts all four buttons in the same places in both examples: the `expand = true` button takes the spare space that the spacer took, and `align_h = "left"` keeps that button at its normal width:

```
gui.HBox {
    -- Same width and background as the spacer example above.
    min_w = 12,
    bgcolor = "#335577",

    -- These buttons will be on the left-hand side of the box
    gui.Button { label = "Cancel" },
    gui.Button { label = "< Back", expand = true, align_h = "left" },

    -- These buttons will be on the right-hand side of the box
    gui.Button { label = "Next >" },
    gui.Button { label = "Confirm" },
}
```

You should not use spacers to center elements as it creates unnecessary boxes, and labels may be slightly off-center (because label widths depend on screen size, DPI, etc and this code doesn't trigger the centering hack). The centering hack is what flow does when a label is centered or stretched by `align_h` or `expand`: it draws the label as a button with no background and no click action, as wide as the whole area, so the client centers the text itself instead of flow guessing the label's width:

```
-- This is bad!
gui.HBox {
    -- Wide enough and colored so that you can see where the middle is.
    min_w = 12,
    bgcolor = "#335577",

    gui.Spacer {},
    gui.Label { label = "I am not properly centered!" },
    gui.Spacer {},
}
```

You should do this instead. The `gui.VBox` around the label is only there to make the example as wide as the one above, you do not need it:

```
gui.VBox {
    min_w = 12,
    bgcolor = "#335577",

    gui.Label { label = "I am centered!", align_h = "center" },
}
```

This applies to other elements as well, because using HBox and Spacer to center elements creates unnecessary containers.

3.2.5 `gui.Nil`

A tool to allow for ternary-ish conditional elements. Unlike an element with `visible = false` (see Hiding Elements), which keeps its space, a `gui.Nil` takes up no space:

Untick the checkbox in this example to remove the box: the last label moves up, because nothing is left behind.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Checkbox { name = "show_box", label = "Show the box", selected = true },
        gui.Label { label = "The box below is only present if the box is ticked:" },
        ctx.form.show_box ~= false and gui.Box { w = 1, h = 1, color = "#FF0000" }
            or gui.Nil {},
        gui.Label { label = "I move up when the box disappears." },
    }
end)
```

Use sparingly, flow still has to process each `Nil` object to be able to know to remove it, and thus could still slow things down. The fastest element is one that doesn't exist, and thus doesn't need processing.

3.2.6 `gui.Container`

A group of elements that you position yourself with `x` and `y` instead of letting flow lay them out. See Manual Positioning.

Example

```
gui.Container {
    w = 6, h = 2, -- Optional
    { type = "box", x = 0, y = 0, w = 1, h = 1, color = "red" },
    { type = "label", x = 1, y = 1.5, label = "Hello world!" },
}
```

3.2.7 `gui.Stack`

This container element places its children on top of each other. All child elements are expanded in both directions.

Note that some elements (such as centred labels) won't pass clicks through to the element below them.

Example

```
gui.Stack {
    min_w = 10,
    gui.Button { label = "Hello world!" },

    -- A red square, drawn on top of the button. The texture is made with a
    -- texture modifier, so this example does not need any files.
    gui.Image { w = 1, h = 1, texture_name = "[fill:16x16:red", padding = 0.2, align_h = "left" },
}
```

3.2.8 `gui.PaginatedVBox`

Like `gui.ScrollableVBox`, but splits the contents into pages instead of scrolling. Only one page is shown at a time, with a `<` button, a "Page 1 of 3" label, and a `>` button underneath to change page. It needs a unique `name`, and `w` and `h` work the same way as for `gui.ScrollableVBox`. Elements are never cut in half: an element that does not fit on a page starts the next one.

`gui.ScrollableVBox` uses this automatically on clients that cannot scroll.

3.2.9 `gui.Flow`

A container with a fixed width that wraps its children onto new rows when they do not fit. You must give it a `w`, and it does not support `h`, `min_w`, or `padding`. Use `spacing` to change the gap between elements (the default is the same as for `gui.VBox`). See Experimental Features for details and an example.

3.2.10 `gui.Padding` (Deprecated)

Wraps a single element to add padding around it. It is deprecated and logs a warning, use the `padding` parameter of the element itself instead, see Padding.

3.3 Functional Elements

3.3.1 `gui.AnimatedImage`

Shows an image that plays an animation. The texture must contain every frame stacked on top of each other (the first frame at the top), with all frames the same size, like the frames of an animated node texture. So a texture with 3 frames of 16x16 pixels is 16 pixels wide and 48 pixels high. `w` and `h` are the size of one displayed frame, and the picture is scaled to that size.

`middle_*` parameters turn on nine-slice drawing, see Nine-Slice Images. The cuts are measured in pixels of one frame.

The examples below use a texture with three frames, a red, a green, and a blue square with a white border. It is made with texture modifiers (`[fill`), so no file is needed: the first `[fill` makes a white 16x48 image, and each one after it paints a 12x12 square into one of the frames. You can use any texture that has the frames stacked in the same way.

Example

```
gui.AnimatedImage {
    w = 2,
    h = 2,

    -- Only useful for styling.
    name = "my_animated_image", -- Optional

    -- Texture that contains all frames, stacked from top to bottom.
    -- This one is 16 pixels wide and 48 high: three frames of 16x16.
    texture_name = "[fill:16x48:white^[fill:12x12:2,2:#E74C3C"
        .. "^[fill:12x12:2,18:#2ECC71^[fill:12x12:2,34:#3498DB",

    -- Number of frames in the texture.
    frame_count = 3,

    -- Milliseconds each frame is shown for. 0 means no animation.
    frame_duration = 500,

    -- Frame to start on, the first frame is 1. Default 1.
    -- This one starts on the green frame.
    frame_start = 2, -- Optional

    -- Nine-slice cuts, in pixels of one frame. Also optional: middle_y,
    -- middle_x2 and middle_y2, see the live example below.
    middle_x = 2, -- Optional
}
```

Dynamic Example

`gui.AnimatedImage` requires a vertically stacked frame strip (normally a PNG file). To keep this example self-contained, it uses Luanti texture modifiers instead:

- `[fill:16x48:white` : Creates a 16x48 white base image.
- `^` : Chains the next modifier to apply over it.
- `[fill:12x12:2,2:#E74C3C` : Paints a 12x12 red square at position (2, 2).

Three 12x12 squares (red, green, blue) are stacked 16 pixels apart at \(y = 2\), \(18\), and \(34\), leaving a 2-pixel white border. (Refer to the Luanti API documentation for more on texture modifiers.)

The example renders three views side-by-side:

1. The full texture strip (`gui.Image`).
2. The cycling animation (`gui.AnimatedImage`).
3. A nine-slice stretched version (`middle_x = 2`).

(Note: Set `frame_duration = 0` to pause the animation on a specific frame using `frame_start` .)

```
-- Three 16x16 frames stacked into one 16x48 texture (red, green, blue).
-- This is a Luanti texture modifier string, not something flow-specific.
-- With a real file you would just write: local FRAMES_TEXTURE = "my_frames.png"
local FRAMES_TEXTURE = "[fill:16x48:white^[fill:12x12:2,2:#E74C3C"
    .. "^[fill:12x12:2,18:#2ECC71^[fill:12x12:2,34:#3498DB"

local form = flow.make_gui(function(player, ctx)
    return gui.HBox {
        gui.VBox {
            gui.Label { label = "The texture" },
            gui.Image { w = 1, h = 3, texture_name = FRAMES_TEXTURE },
        },
        gui.VBox {
            gui.Label { label = "One frame at a time" },
            gui.AnimatedImage {
                w = 3, h = 3,
                texture_name = FRAMES_TEXTURE,
                frame_count = 3,
                frame_duration = 500,
            },
        },
        gui.VBox {
            gui.Label { label = "Nine-slice (middle_x = 2)" },
            gui.AnimatedImage {
                w = 4, h = 2,
                texture_name = FRAMES_TEXTURE,
                frame_count = 3,
                frame_duration = 500,
                middle_x = 2,
            },
        },
    }
end)
```

3.3.2 `gui.Background`

Draws a picture behind everything else in the form: all backgrounds are drawn before any other element, whatever order they are in. The picture (`texture_name`) is stretched to `w` by `h` . If `auto_clip` is `true` , it is cut off at the edge of the form instead and `w` and `h` are ignored.

You usually do not need this element, set `bgimg` on a container or the root element instead (see Padding). Inside a `gui.Stack` , a `gui.ScrollableVBox` , or an embedded form, flow draws backgrounds without `auto_clip` as ordinary images.

Example

```
gui.Background {
    w = 6,
    h = 3,

    -- Picture to draw. This one is a white border around a blue square,
    -- made with texture modifiers so that no file is needed.
    texture_name = "[fill:16x16:white^[fill:12x12:2,2:#336699",

    -- Cut off at the form edge, ignore w and h.
    auto_clip = false, -- Optional
}
```

Live Example

A background is only useful when something is drawn on top of it, so this example puts one in a `gui.Container` (see Manual Positioning) together with a button. The background comes after the button in the list, but it is still drawn behind it.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Container {
            w = 6, h = 3,
            { type = "button", x = 1.5, y = 1, w = 3, h = 1, label = "In front" },
            {
                type = "background",
                x = 0, y = 0, w = 6, h = 3,
                texture_name = "[fill:16x16:white^[fill:12x12:2,2:#336699",
            },
        },
    }
end)
```

3.3.3 `gui.Background9`

Like `gui.Background` , but the picture is drawn as a nine-slice so that it can be stretched to any size without distorting its corners.

Example

```
gui.Background9 {
    w = 6,
    h = 3,

    -- Picture to draw. This one is a white border around a blue square,
    -- made with texture modifiers so that no file is needed.
    texture_name = "[fill:16x16:white^[fill:12x12:2,2:#336699",

    -- Cut off at the form edge, ignore w and h.
    auto_clip = false,

    -- Nine-slice cut, see "Nine-slice images". The border of the texture is
    -- 2 pixels wide, so cut 2 pixels from every side.
    middle_x = 2,
    middle_y = 2, -- Optional
    middle_x2 = -2, -- Optional
    middle_y2 = -2, -- Optional
}
```

Live Example

Both backgrounds use the same texture and are the same size. The left one is a plain `gui.Background` that stretches the whole picture, so its border gets thick and blurry. The right one is a `gui.Background9` with `middle_x = 2`, so only the middle is stretched and the border stays sharp.

```
local FRAME = "[fill:16x16:white^[fill:12x12:2,2:#336699"

local form = flow.make_gui(function(player, ctx)
    return gui.HBox {
        gui.Container {
            w = 5, h = 3,
            { type = "label", x = 0.5, y = 1.5, label = "Background" },
            { type = "background", x = 0, y = 0, w = 5, h = 3, texture_name = FRAME },
        },
        gui.Container {
            w = 5, h = 3,
            { type = "label", x = 0.5, y = 1.5, label = "Background9" },
            {
                type = "background9",
                x = 0, y = 0, w = 5, h = 3,
                texture_name = FRAME,
                middle_x = 2,
            },
        },
    }
end)
```

3.3.4 `gui.Bgcolor`

Sets the colors behind the form. This does the same thing as the `bgcolor`, `fbgcolor`, and `bg_fullscreen` options of the root element, which are explained in Experimental Features and are easier to use. `fullscreen` can be `false`, `true`, `"both"`, or `"neither"`.

Example

```
gui.VBox {
    gui.Bgcolor {
        -- Background color of the form.
        bgcolor = "#336699",

        -- false, true, "both", or "neither".
        fullscreen = false, -- Optional

        -- Color of the area behind the form.
        fbgcolor = "#00000080", -- Optional
    },

    gui.Label { label = "The form has a blue background." },
}
```

Live Example

Pick a color to change the background of the form.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Use the picked color (the first item before anything is picked).
        gui.Bgcolor { bgcolor = ctx.form.color or "red" },

        gui.Label { label = "Pick a background color:" },
        gui.Dropdown {
            name = "color",
            items = { "red", "green", "blue", "purple" },
        },
    }
end)
```

3.3.5 `gui.Box`

Draws a solid rectangle of one color. Boxes have no natural size, so give them a `w` and `h`.

Unlike everywhere else, a hexadecimal color without an alpha value (`"#RGB"` or `"#RRGGBB"`) is drawn with an alpha of `8C` (140 out of 255, about 55% opaque) here. Add the alpha value (`"#FF0000FF"`) to make it opaque. Color names such as `"red"` are not affected and are opaque unless you add an alpha value (`"red#80"`). Flow also draws the `bgcolor` of `gui.HBox`, `gui.VBox`, and other containers with a box, so the same applies to it.

Example

```
gui.Box {
    w = 1,
    h = 2,

    -- Fill color.
    color = "#FF0000",
}
```

Opaque and Translucent Colors

These four boxes all use red, written in different ways. The first and last ones are translucent and let the background of the form show through, and the other two are fully opaque:

```
gui.HBox {
    -- #RRGGBB gets an alpha of 8C here, so this one is translucent.
    gui.Box { w = 1, h = 2, color = "#FF0000" },

    -- Add the alpha value to make it opaque.
    gui.Box { w = 1, h = 2, color = "#FF0000FF" },

    -- Color names are opaque unless you add an alpha value.
    gui.Box { w = 1, h = 2, color = "red" },

    -- This one has an alpha value of 80 (about half).
    gui.Box { w = 1, h = 2, color = "red#80" },
}
```

3.3.6 `gui.Button`

A clickable button showing `label`. Your `on_event` function is called when it is clicked. If you do not set `w` and `h`, the button is sized to fit the label (but never smaller than 1.6 by 0.8).

Buttons do not store a value in `ctx.form`.

Example

```
gui.Button {
    w = 6, -- Optional
    h = 2, -- Optional
    name = "my_button", -- Optional
    label = "Hello world!", -- Optional
}
```

3.3.7 `gui.ButtonExit`

A button that closes the form when it is clicked. Your `on_event` function is still called before the form closes. In all other respects it works like `gui.Button`.

Example

```
gui.ButtonExit {
    w = 6, -- Optional
    h = 2, -- Optional
    name = "my_button_exit", -- Optional
    label = "Hello world!", -- Optional
}
```

3.3.8 `gui.ButtonURL`

A button that, when clicked, offers the player the choice of opening `url` in their web browser. `url` must start with `http://` or `https://`. Your `on_event` function is also called. In all other respects it works like `gui.Button`.

Example

```
gui.ButtonURL {
    w = 6, -- Optional
    h = 2, -- Optional
    name = "my_button_url", -- Optional
    label = "Hello world!", -- Optional

    -- Web address to offer to open.
    url = "https://example.com",
}
```

3.3.9 `gui.ButtonUrlExit`

Like `gui.ButtonURL`, but the form also closes when the button is clicked. Unlike `gui.ButtonURL`, it does not get a size from its label, so `w` and `h` are required.

Example

```
gui.ButtonUrlExit {
    w = 6,
    h = 2,
    name = "my_button_url_exit", -- Optional
    label = "Hello world!", -- Optional

    -- Web address to offer to open.
    url = "https://example.com",
}
```

3.3.10 `gui.Checkbox`

A tick box with a text `label`. `selected` sets whether it starts ticked. If you give it a `name`, `ctx.form[name]` is `true` or `false` and is updated as soon as the player ticks or unticks it.

Checkboxes are sized automatically and do not accept `w` or `h`.

Example

```
gui.Checkbox {
    name = "my_checkbox", -- Optional
    label = "Hello world!", -- Optional
    selected = true, -- Optional
}
```

3.3.11 `gui.Dropdown`

A drop-down list that shows the currently selected item and lets the player pick another one from `items` (a list of strings). `selected_idx` is the number of the item that is selected at first (the first item is `1`).

`ctx.form[name]` is the text of the selected item. If `index_event = true` it is the number of the selected item instead, which is easier to work with when two items have the same text or when the text is translated.

If you do not set `w` and `h`, the dropdown is sized to fit its longest item (but never smaller than 2 by 0.8). Item text cannot be coloured.

Example

```
gui.Dropdown {
    w = 4, -- Optional
    h = 0.8, -- Optional
    name = "my_dropdown", -- Optional

    -- List of choices.
    items = { "Apple", "Banana", "Cherry" },

    -- Number of the item selected at first.
    selected_idx = 2, -- Optional

    -- Store the item number in ctx.form instead of its text.
    index_event = false, -- Optional
}
```

3.3.12 `gui.Field`

A single-line text box. `label`, if set, is shown above the box (flow leaves room for it automatically). `default` is the text it starts with.

`ctx.form[name]` is the text in the box. Newlines and other control characters are removed from it, but tabs are kept (a player can only get a newline into a field by pasting). See Security for the exact rules. The text is sent to the server when the player presses a button or Enter. To react to Enter, see Handling Enter Keypresses in Fields.

If you do not set `w` and `h`, the field is sized to fit its default text and label (but never smaller than 3 by 0.8).

Example

```
gui.Field {
    w = 6, -- Optional
    h = 2, -- Optional
    name = "my_field", -- Optional
    label = "Hello world!", -- Optional

    -- Text the field starts with.
    default = "Hello world!", -- Optional

    -- Called when the player presses Enter.
    on_key_enter = function(player, ctx) end, -- Optional

    -- Close the form on Enter. Default false.
    close_on_enter = false, -- Optional

    -- Make "Done" on the mobile on-screen keyboard act like Enter.
    enter_after_edit = false, -- Optional
}
```

3.3.13 `gui.FieldCloseOnEnter`

Sets whether pressing Enter in the field with this `name` closes the form. You normally do not need this element, as `gui.Field` has a `close_on_enter` parameter (and by default Enter does not close the form).

Example

```
gui.VBox {
    gui.Field { name = "my_field_close_on_enter", label = "Press Enter to close the form" },

    gui.FieldCloseOnEnter {
        -- Name of the field.
        name = "my_field_close_on_enter",
        close_on_enter = true,
    },
}
```

Live Example

Type something and press Enter. The label shows the text that was sent. While the checkbox is ticked, Enter also closes the form.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Field { name = "note", label = "Type something, then press Enter:" },

        -- Close on Enter only while the box is ticked.
        gui.FieldCloseOnEnter {
            name = "note",
            close_on_enter = ctx.form.close or false,
        },

        gui.Checkbox { name = "close", label = "Close the form on Enter" },
        gui.Label { label = "Last text sent: " .. (ctx.form.note or "") },
    }
end)
```

3.3.14 `gui.FieldEnterAfterEdit`

Experimental, and only affects Android. If `enter_after_edit` is `true`, pressing "Done" in the text input dialog of the field with this `name` counts as pressing Enter. You normally do not need this element, as `gui.Field` has an `enter_after_edit` parameter.

Example

```
gui.VBox {
    gui.Field { name = "my_field_enter_after_edit", label = "Edit me, then press Done" },

    gui.FieldEnterAfterEdit {
        -- Name of the field.
        name = "my_field_enter_after_edit",
        enter_after_edit = true,
    },
}
```

Live Example

Press Enter in the field and the counter goes up. On Android, tick the checkbox to make "Done" in the on-screen keyboard count as Enter too.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Field {
            name = "msg",
            label = "Edit, then press Enter (or Done on Android):",
            on_key_enter = function(player, ctx)
                ctx.enters = (ctx.enters or 0) + 1
                return true -- redraw
            end,
        },

        -- Count "Done" as Enter only while the box is ticked.
        gui.FieldEnterAfterEdit {
            name = "msg",
            enter_after_edit = ctx.form.after_edit or false,
        },

        gui.Checkbox { name = "after_edit", label = "Done counts as Enter (Android)" },
        gui.Label { label = "Enter pressed " .. (ctx.enters or 0) .. " times" },
    }
end)
```

3.3.15 `gui.Hypertext`

Shows text that can be formatted (bold, colors, different sizes, images, ...) and can contain clickable links. The text is written in a small HTML-like language, which is described in Hypertext Markup. `w` and `h` are the size of the area that the text is wrapped to fit in.

If the markup contains `<action name=...>` links, `ctx.form[name]` is set to `"action:"` followed by the action name when the player clicks one. This behaviour is experimental and may change.

Example

```
gui.Hypertext {
    w = 6,
    h = 2,

    -- Receives the clicked action, see above.
    name = "my_hypertext", -- Optional

    -- Formatted text, see "Hypertext markup".
    text = "<big>Hello</big> <b>bold</b> <i>italic</i> " ..
        "<style color='red'>red</style> world!",
}
```

3.3.16 `gui.Image`

Shows a picture, stretched to fit `w` by `h`. If you want to use one small texture for pictures of many sizes, use the `middle_*` parameters, see Nine-Slice Images.

Example

```
gui.Image {
    w = 4,
    h = 2,

    -- Texture to draw. This one is a white border around a blue square,
    -- made with texture modifiers so that no file is needed.
    texture_name = "[fill:16x16:white^[fill:12x12:2,2:#336699",

    -- Nine-slice cuts, in pixels of the texture. The border is 2 pixels
    -- wide, so these keep it the same width however big the image is.
    middle_x = 2, -- Optional
    middle_y = 2, -- Optional
    middle_x2 = -2, -- Optional
    middle_y2 = -2, -- Optional
}
```

3.3.17 `gui.ImageButton`

A button that is drawn as a picture instead of the usual button face. The picture (`texture_name`) is stretched to `w` by `h`, and `label` is drawn on top of it. It works like `gui.Button` otherwise.

- `noclip`: Allow the button to be drawn outside the form's area.
- `drawborder`: Draw the normal button border around the picture.
- `pressed_texture_name`: Picture to show while the button is held down.

Example

```
gui.ImageButton {
    w = 6,
    h = 2,

    -- Picture to draw as the button. This one is a blue square, made with
    -- a texture modifier so that no file is needed.
    texture_name = "[fill:16x16:#3498DB",
    name = "my_image_button", -- Optional
    label = "Hello world!", -- Optional
    noclip = false, -- Optional
    drawborder = false, -- Optional

    -- Picture to show while the button is held down (a red square).
    pressed_texture_name = "[fill:16x16:#E74C3C", -- Optional
}
```

3.3.18 `gui.ImageButtonExit`

Like `gui.ImageButton`, but the form also closes when the button is clicked.

Example

```
gui.ImageButtonExit {
    w = 6,
    h = 2,

    -- Picture to draw as the button. This one is a blue square, made with
    -- a texture modifier so that no file is needed.
    texture_name = "[fill:16x16:#3498DB",
    name = "my_image_button_exit", -- Optional
    label = "Hello world!", -- Optional
    noclip = false, -- Optional
    drawborder = false, -- Optional

    -- Picture to show while the button is held down (a red square).
    pressed_texture_name = "[fill:16x16:#E74C3C", -- Optional
}
```

3.3.19 `gui.ItemImage`

Shows the picture that is used for an item or node in inventories (`item_name`, such as `"unifieddyes:colorium"`), stretched to fit `w` by `h`.

Example

```
gui.ItemImage {
    w = 2,
    h = 2,

    -- Registered item or node name.
    item_name = "unifieddyes:colorium",
}
```

3.3.20 `gui.ItemImageButton`

A button that shows the inventory picture of an item or node (`item_name`). By default, the item's description is shown as a tooltip when the player hovers over the button, which you can replace with the `tooltip` parameter. It works like `gui.Button` otherwise.

Give every item image button its own unique `name`, otherwise its tooltip may not work.

Example

```
gui.ItemImageButton {
    w = 3,
    h = 2,

    -- Registered item or node name.
    item_name = "unifieddyes:colorium",
    name = "my_item_image_button", -- Optional
    label = "Hello world!", -- Optional
}
```

3.3.21 `gui.Label`

Shows text. Labels do not wrap long lines automatically: use `\n` to start a new line and do not use labels for long paragraphs. For those, use `gui.Hypertext`, or a `gui.Textarea` without a `name` (which wraps its text and gets a scrollbar if the text is too long).

Labels are sized to fit their text, and flow ignores color escape sequences when it estimates the width. To make the text bigger or smaller, use the `font_size` style (for example `style = {font_size = "*2"}`), see Styling.

Example

```
gui.Label {
    label = "Hello world!",
}
```

3.3.22 `gui.List`

Shows a grid of inventory slots, with the items of one inventory list in it. This is how you show the player's inventory, a chest, a furnace's input slots, and so on.

- `inventory_location`: Which inventory to show. Common values are `"current_player"` (the inventory of the player who has the form open), `"nodemeta:X,Y,Z"` (the inventory of the node at that position, for example `"nodemeta:10,5,-3"`), and `"detached:name"` (a detached inventory). `"player:name"` is the inventory of a specific player and is only ever sent to that player.
- `list_name`: The name of the list inside that inventory, such as `"main"` or `"craft"`.
- `w` and `h`: How many slots wide and high to draw (whole numbers).

Slots are filled from left to right, then top to bottom, and the player can move items in and out of them. The list is only shown if the inventory has been sent to the player's client. Only as many slots as the inventory list has are drawn, even if `w` times `h` is larger.

Each slot is 1 by 1, with 0.25 between slots, so a list that is 8 slots wide and 4 high is 9.75 by 4.75 big. To change the slot size or spacing, use the `size` and `spacing` styles, see Styling.

Example

```
gui.List {
    -- Which inventory to show.
    inventory_location = "current_player",

    -- Which list in that inventory to show.
    list_name = "main",
    w = 4,
    h = 2,

    -- Index of the first slot to draw. Unlike most numbers here, this starts
    -- at 0. Default 0.
    starting_item_index = 3, -- Optional
}
```

3.3.23 `gui.Listcolors`

Sets the colors of inventory slots. It is invisible, takes up no space, and should come before the `gui.List` elements it is for.

- `slot_bg_normal` : Background color of a slot.
- `slot_bg_hover` : Background color of a slot while the mouse is over it.
- `slot_border` : Color of the border of the slots.
- `tooltip_bgcolor` : Default background color of item tooltips.
- `tooltip_fontcolor` : Default text color of item tooltips.

Only the first two are required. To use pictures instead of colors for slots, see Unsupported Features.

Example

```
gui.VBox {
    gui.Listcolors {
        -- Slot background color.
        slot_bg_normal = "#2E7D32",

        -- Slot background color when hovered.
        slot_bg_hover = "#66BB6A",

        -- Slot border color.
        slot_border = "#FFFFFF", -- Optional

        -- Default tooltip background color.
        tooltip_bgcolor = "#000000", -- Optional

        -- Default tooltip text color.
        tooltip_fontcolor = "#FFFF00", -- Optional
    },

    gui.List {
        inventory_location = "current_player",
        list_name = "main",
        w = 4, h = 1,
    },
}
```

Live Example

Pick a theme, then hover over the slots to see both colors.

```
local themes = {
    Green = { normal = "#2E7D32", hover = "#66BB6A" },
    Blue = { normal = "#1565C0", hover = "#64B5F6" },
    Red = { normal = "#B71C1C", hover = "#EF5350" },
}

local form = flow.make_gui(function(player, ctx)
    local theme = themes[ctx.form.theme or "Green"]
    return gui.VBox {
        gui.Dropdown { name = "theme", items = { "Green", "Blue", "Red" } },
        gui.Label { label = "Hover over a slot:" },

        -- Color the lists that come after this.
        gui.Listcolors {
            slot_bg_normal = theme.normal,
            slot_bg_hover = theme.hover,
            slot_border = "#FFFFFF",
        },
        gui.List {
            inventory_location = "current_player",
            list_name = "main",
            w = 8, h = 1,
        },
    }
end)
```

3.3.24 `gui.Listring`

Sets where items go when the player shift-clicks them. A *ring* is an ordered list of inventory lists that Luanti keeps for the form, which starts out empty. Every `gui.Listring{inventory_location = ..., list_name = ...}` adds one list to the end of the ring. When the player shift-clicks an item in a list that is in the ring, the item is sent to the next list in the ring. Lists that are not in the ring keep Luanti's default shift-click behavior.

For example, after the two `gui.Listring` elements in the example below the ring is `main`, then `craft`. Shift-clicking an item in `main` sends it to `craft`, and shift-clicking an item in `craft` sends it to `main`.

A list can be added to the ring more than once. When the player shift-clicks an item, Luanti looks for the first entry in the ring that is the list the item is in, and sends the item to the entry after it (the first entry comes after the last one). Later entries of the same list are only destinations: they never decide where items from that list go.

For example, shift-clicking in a ring of `main`, `craft`, `main` sends items from `main` to `craft` and items from `craft` to `main`, the same as a ring of `main` and `craft`. Repeating a list is useful when several lists should send to the same one. In a ring of `main`, `craft`, `main`, `output`, items from `main` still go to `craft`, items from `craft` go to the second `main`, and items from `output` go to the first `main` (the ring wraps around), so both `craft` and `output` send to `main`.

`gui.Listring{}` with no parameters adds the last two `gui.List` elements that came before it. It is invisible and takes up no space.

Example

```
gui.VBox {
    gui.List { inventory_location = "current_player", list_name = "main", w = 4, h = 1 },
    gui.List { inventory_location = "current_player", list_name = "craft", w = 3, h = 1 },

    -- Shift-clicking an item in one of the lists above moves it to the
    -- other one, because both of them are in the ring.
    gui.Listring {
        -- Same as for gui.List.
        inventory_location = "current_player", -- Optional

        -- Same as for gui.List.
        list_name = "main", -- Optional
    },
    gui.Listring {
        inventory_location = "current_player",
        list_name = "craft",
    },
}
```

Live Example

Shift-click an item in either list to send it to the other one. Untick the checkbox to remove the ring and shift-click goes back to Luanti's default.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Checkbox { name = "ring", label = "Link the lists", selected = true },
        gui.Label { label = "Shift-click an item:" },
        gui.List {
            inventory_location = "current_player",
            list_name = "main",
            w = 8, h = 1,
        },
        gui.List {
            inventory_location = "current_player",
            list_name = "craft",
            w = 3, h = 3,
        },

        -- Link the two lists above.
        ctx.form.ring ~= false and gui.Listring {} or gui.Nil {},
    }
end)
```

3.3.25 `gui.Model`

Shows a 3D model that the player can turn around with their mouse.

- `mesh` : File name of the model, from the `models` folder of any loaded mod.

- `textures` : The textures for the model, one for each of the model's materials in order.
- `rotation_x` and `rotation_y` : How far the model is turned at first, in degrees.
- `continuous` : Keep the model rotating by itself.
- `mouse_control` : Whether the player can rotate the model with their mouse. Defaults to `true` .
- `frame_loop_begin` and `frame_loop_end` : The range of animation frames to play. Defaults to all frames.
- `animation_speed` : Frames per second to play the animation at. Defaults to `0` (no animation).

`name` is optional and only useful for styling.

Example

```
gui.Model {
    w = 5,
    h = 5,
    padding = 0.5,

    -- Only useful for styling.
    name = "my_model", -- Optional

    -- Model file name.
    mesh = "voxelmodel.obj",

    -- One texture for each material of the model. This one is a blue
    -- square, made with a texture modifier so that no file is needed.
    textures = { "[fill:16x16:#3498DB^[fill:8x8,0,4:#888888" },

    -- Starting rotation in degrees. This tilts the model so that you can
    -- see it is 3D.
    rotation_x = -20, -- Optional

    -- Starting rotation in degrees.
    rotation_y = 30, -- Optional

    -- Keep rotating by itself. Default false.
    continuous = true, -- Optional

    -- Let the player rotate the model. Default true.
    mouse_control = true, -- Optional

    -- First animation frame to play.
    frame_loop_begin = 5, -- Optional

    -- Last animation frame to play.
    frame_loop_end = 6, -- Optional

    -- Frames per second. Default 0.
    animation_speed = 7, -- Optional
}
```

3.3.26 `gui.Pwdfield`

A single-line text box for passwords: what the player types is hidden. It works like `gui.Field`, apart from not having a `default`. `ctx.form[name]` is the text that was typed. `on_key_enter` is supported, see Handling Enter Keypresses in Fields.

Example

```
gui.Pwdfield {
    w = 6, -- Optional
    h = 2, -- Optional
    name = "my_pwdfield", -- Optional
    label = "Hello world!", -- Optional

    -- Called when the player presses Enter.
    on_key_enter = function(player, ctx) end, -- Optional
}
```

3.3.27 `gui.ScrollContainer`

An area of a fixed size that clips the elements inside it and moves them when a scrollbar is moved. You usually want `gui.ScrollableVBox` instead, which does all of this for you. The elements inside are positioned by hand with `x` and `y`, like in `gui.Container`.

- `w` and `h`: The size of the visible area.
- `scrollbar_name`: The `name` of the `gui.Scrollbar` that controls the area. You need to add that scrollbar yourself.
- `orientation`: `"vertical"` (the default) or `"horizontal"`.
- `scroll_factor`: How far the contents move for each step of the scrollbar's value. Default `0.1`. It cannot be negative.

Flow cannot work out the range of the scrollbar for you here. Set `max` and `thumbsize` with `gui.ScrollbarOptions` before the scrollbar:

- `max` is the height that is hidden divided by `scroll_factor`. The hidden height is the height of the contents minus the `h` of the container. The scrollbar then runs from `0` to `max`, and each step moves the contents by `scroll_factor`.
- `thumbsize` is the size of the draggable thumb, counted in the same steps. Luanti defines it as the number of steps the thumb spans out of the whole range, so the thumb is as large a fraction of the track as the visible part is of the contents: `thumbsize = max * h / content_height`.

Both must be whole numbers, so round the result.

Example

```
gui.HBox {
    gui.ScrollContainer {
        w = 4, h = 3,
        scrollbar_name = "my_scrollbar",
        orientation = "vertical",
        { type = "label", x = 0, y = 0.5, label = "Top" },
        { type = "label", x = 0, y = 7.5, label = "Bottom" },
    },

    -- The contents are 8 high and 3 are visible, so 5 are hidden. This
    -- is 50 scroll steps of 0.1 each (max = 5 / 0.1).
    -- thumbsize = 50 * 3 / 8 = 18.75, rounded to 19.
    gui.ScrollbarOptions { opts = { max = "50", thumbsize = "19" } },
    gui.Scrollbar {
        w = 0.5, h = 3,
        orientation = "vertical",
        name = "my_scrollbar",
    },
}
```

3.3.28 `gui.Scrollbar`

A slider that the player can drag. `ctx.form[name]` is its position, a whole number from `min` to `max` (by default `0` to `1000`, change them with `gui.ScrollbarOptions` before the scrollbar). `value` is the position it starts at, and `orientation` is `"vertical"` or `"horizontal"`. Use it with a `gui.ScrollContainer`, or by itself as a slider.

Example

```
gui.Scrollbar {
    w = 0.6,
    h = 4,

    -- "vertical" or "horizontal".
    orientation = "vertical",
    name = "my_scrollbar", -- Optional

    -- Starting position, from 0 to 1000 by default.
    value = 300, -- Optional
}
```

Do not use the `scrollbar_bg`, `slider`, `arrow_up`, and `arrow_down` texture parameters: Luanti has no such parameters, and on Luanti 5.8 and newer a scrollbar that sets any of them is not shown at all. See Scrollbar Textures Do Not Work.

3.3.29 `gui.ScrollbarOptions`

Sets the options of the `gui.Scrollbar` elements that come after it. It is invisible and takes up no space. `opts` is a table with any of these options, all values are strings that contain whole numbers (except `arrows`):

- `min`: The smallest value (default `0`).
- `max`: The largest value (default `1000`). If it is the same as `min`, the scrollbar is disabled.
- `smallstep`: How much the value changes when an arrow is clicked or the mouse wheel is scrolled (default `10`). A negative number resets it to `10`.
- `largestep`: How much the value changes for Page Up and Page Down (default `100`). A negative number resets it to `100`.
- `thumbsize`: The size of the thumb (the part that is dragged), counted in values out of the range of the scrollbar. For a scrollbar from 1 to 100, a `thumbsize` of `10` makes the thumb a tenth of the scrollbar long. If it is zero or less, it is reset to `1`.
- `arrows`: `"show"` or `"hide"` the arrow buttons. `"default"` hides them if the scrollbar gets too small, and shows them otherwise.

Example

```
gui.VBox {
    -- Changes the scrollbar below: it goes from 0 to 100, has no arrows, and
    -- moves in steps of 5.
    gui.ScrollbarOptions {
        opts = { min = "0", max = "100", smallstep = "5", arrows = "hide" },
    },

    gui.Scrollbar {
        name = "my_scrollbar",
        orientation = "horizontal",
        w = 6, h = 0.6,
        value = 50,
    },
}
```

Live Example

Drag the scrollbar or click the arrows. The value is limited to 0-100 and changes in steps of 5.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = "Volume: " .. (ctx.form.volume or 0) },

        -- Set the range and steps of the scrollbar below.
        gui.ScrollbarOptions {
            opts = {
                min = "0", max = "100",
                smallstep = "5", largestep = "20",
                arrows = ctx.form.arrows and "show" or "hide",
            },
        },
        gui.Scrollbar {
            name = "volume",
            orientation = "horizontal",
            w = 6, h = 0.6,
        },

        gui.Checkbox { name = "arrows", label = "Show arrow buttons" },
    }
end)
```

3.3.30 `gui.SetFocus`

Gives keyboard focus to the element with this `name`. Put it before that element. Normally the focus is not changed when the form is shown again (for example by `form:update()`), so that the focus the player chose is kept. Set `force` to `true` to move the focus every time.

Checkboxes, buttons (of all kinds), tables, text lists, dropdowns, fields, password fields, text areas, and scrollbars can be focused.

Example

```
gui.VBox {
    gui.Field { name = "other_field", label = "Not focused" },

    gui.SetFocus {
        -- Name of the element to focus.
        name = "my_set_focus",

        -- Move the focus every time. Default false.
        force = false, -- Optional
    },
    gui.Field { name = "my_set_focus", label = "Focused" },
}
```

Live Example

Pick a field in the dropdown and the keyboard focus jumps to it. Start typing to see which field has the focus.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Move the focus to the picked field on every redraw.
        gui.SetFocus { name = ctx.form.target or "first", force = true },

        gui.Dropdown { name = "target", items = { "first", "second", "third" } },
        gui.Field { name = "first", label = "First field" },
        gui.Field { name = "second", label = "Second field" },
        gui.Field { name = "third", label = "Third field" },
    }
end)
```

3.3.31 `gui.Style`

Sets style properties of elements by their `name`. See Styling. It is invisible and takes up no space.

Example

```
gui.VBox {
    gui.Style {
        selectors = { "my_button" },
        props = { bgcolor = "red", textcolor = "yellow" },
    },

    gui.Button { name = "my_button", label = "Styled button" },
    gui.Button { name = "other_button", label = "Unstyled button" },
}
```

Live Example

Pick a color to tint the button named "fancy". The other button is not affected.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Dropdown { name = "color", items = { "red", "green", "blue" } },

        -- Style only the element named "fancy".
        gui.Style {
            selectors = { "fancy" },
            props = { bgcolor = ctx.form.color or "red", textcolor = "yellow" },
        },
        gui.Button { name = "fancy", label = "Styled button" },
        gui.Button { name = "plain", label = "Plain button" },
    }
end)
```

3.3.32 `gui.StyleType`

Sets style properties of every element of a type. See Styling. It is invisible and takes up no space.

Example

```
gui.VBox {
    gui.StyleType {
        selectors = { "button", "image_button" },
        props = { textcolor = "yellow" },
    },

    gui.Button { label = "Yellow text" },
    gui.Label { label = "Labels are not buttons." },
}
```

Live Example

Pick a text color. Every button after the style changes, but the label does not.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Dropdown { name = "color", items = { "yellow", "orange", "cyan" } },

        -- Color the text of every button that comes after this.
        gui.StyleType {
            selectors = { "button" },
            props = { textcolor = ctx.form.color or "yellow" },
        },
        gui.Button { label = "Button A" },
        gui.Button { label = "Button B" },
        gui.Label { label = "I am a label, so I am not affected." },
    }
end)
```

3.3.33 `gui.Tabheader`

A row of tabs. `captions` is the text of each tab, `ctx.form[name]` is the number of the selected tab (the first tab is `1`), and `current_tab` is the tab that is selected at first. Showing the right contents for the selected tab is up to you, usually by reading `ctx.form[name]` in your build function.

Give it both a `w` and an `h`: flow cannot work out how much space it needs. If `transparent` is `true` the tabs are semi-transparent, and `draw_border` draws a thin line along the bottom of the tabs.

Example

```
gui.Tabheader {
    w = 6,
    h = 0.8,
    name = "my_tabheader", -- Optional

    -- Text of each tab.
    captions = { "General", "Video", "Audio" },

    -- Tab selected at first, the first is 1.
    current_tab = 2, -- Optional

    -- Draw semi-transparent. Default false.
    transparent = false, -- Optional

    -- Draw a line along the tabs. Default false.
    draw_border = false, -- Optional
}
```

3.3.34 `gui.Table`

A table with rows and columns that scrolls if it has too many rows. The player can select one row, and `ctx.form[name]` is the number of the selected row (the first row is `1`). `selected_idx` is the row that is selected at first.

`cells` is one flat list with the text of every cell, going left to right along each row and then down to the next row. The number of columns comes from `gui.TableColumns` (the default is one text column). Put `gui.TableColumns` and `gui.TableOptions` before the table, as they only affect the table that comes after them. They do not take up any space.

Tables have no natural size, so you must give them a `w` and `h`.

Example

```
gui.Table {
    w = 4,
    h = 2,
    name = "my_table", -- Optional

    -- Text of every cell, row by row. There is one column by default, so
    -- this is four rows.
    cells = { "Alice", "Bob", "Carol", "Dave" },

    -- Row selected at first, the first row is 1.
    selected_idx = 3, -- Optional
}
```

Example With Columns and Options

This example uses `gui.TableColumns` to set the width and alignment of two text columns and `gui.TableOptions` to change the color of the selected row. Both are explained below.

```
gui.VBox {
    gui.TableColumns {
        tablecolumns = {
            { type = "text", opts = { width = "8" } },
            { type = "text", opts = { align = "right" } },
        },
    },
    gui.TableOptions {
        opts = { highlight = "#335577" },
    },
    gui.Table {
        w = 10, h = 4,
        name = "scores",
        cells = {
            "Alice", "10",
            "Bob", "7",
        },
    },
}
```

3.3.35 `gui.TableColumns`

Sets the columns of the `gui.Table` that follows it. `tablecolumns` is a list with one `{type = ..., opts = {...}}` entry for each column, from left to right. `type` is one of:

- `"text"` : Show the cell contents as text.
- `"image"` : The cell contents are a number that picks an image, which you set with the options `["0"] = "texture.png"` , `["1"] = "other.png"` , and so on (the numbers do not have to be consecutive). Empty cells and cells that are not numbers use image `0` . Images are scaled down to fit the height of the row if necessary.
- `"color"` : The cell contents are a color that is used for the text of the next column(s). Use the `span` option to set how many columns it affects (default: all remaining columns).
- `"indent"` : The cell contents are a number that sets how far the next column is indented.
- `"tree"` : Like `"indent"` , but the player can open and close the indented rows, like in a file browser.

`opts` is a table of options, all values are strings. `width` and `padding` are measured in "em" units (relative to the font size), unlike most other sizes in flow, which are in slot units:

- `align` : For text and image columns, where the content goes in the cell: `"left"` (default), `"center"` , `"right"` , or `"inline"` .
- `width` : For text and image columns, the minimum width, in "em" units (relative to the font size, default `0`). For indent and tree columns, how far each level is indented (default `1.5`).
- `padding` : Space to the left of the column, in "em" units (default `0.5` , or `0` for indent columns).

- `tooltip` : Text to show when hovering over the column.
- `span` : For color columns, how many of the following columns the color affects (default: all remaining columns).

Example

See Example With Columns and Options under `gui.Table` , which uses the same two text columns. The live example below also uses a color column. For indent and tree columns, see Example With Indent and Tree Columns.

Live Example

Pick an alignment for the last column and select a row. The middle column colors the text of the last one.

```
local names = { "Alice", "Bob", "Carol" }

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Dropdown { name = "align", items = { "left", "center", "right", "inline" } },

        -- Set the columns of the table below.
        gui.TableColumns {
            tablecolumns = {
                { type = "text", opts = { width = "8" } },
                { type = "color", opts = {} },
                { type = "text", opts = { align = ctx.form.align or "left" } },
            },
        },
        gui.Table {
            w = 6, h = 3,
            name = "scores",
            cells = {
                "Alice", "#66FF66", "10",
                "Bob", "#FFFF66", "7",
                "Carol", "#FF6666", "3",
            },
        },

        gui.Label { label = "Selected: " .. names[ctx.form.scores or 1] },
    }
end)
```

Example With Indent and Tree Columns

An `"indent"` or `"tree"` column takes one cell in every row, like any other column, but the cell is not shown. It holds the level of the row: `"0"` for the leftmost level, `"1"` for the next one, and so on (a negative number counts as

`0`). The column after it is moved to the right by the level times the `width` option of the indent or tree column, so put the text you want to show in that next column.

```
gui.VBox {
    gui.TableColumns {
        tablecolumns = {
            -- The level of each row. It is not shown.
            { type = "tree", opts = {} },
            -- The text of each row.
            { type = "text", opts = { width = "8" } },
        },
    },
    gui.TableOptions {
        -- Open the rows of level 0 at first.
        opts = { opendepth = "1" },
    },
    gui.Table {
        w = 5, h = 3,
        name = "files",
        cells = {
            "0", "Fruit",
            "1", "Apple",
            "1", "Banana",
            "0", "Vegetables",
            "1", "Carrot",
        },
    },
}
```

Each row has two cells: its level, then its text. `Apple` and `Banana` are one level deeper than `Fruit`, so they are indented and belong to it, and `Carrot` belongs to `Vegetables`. A row is the parent of the rows right below it that are deeper than it.

With `"tree"`, a parent row shows a `-` (open) or `+` (closed) before its text, and rows without children show no sign. The player clicks the sign, or double-clicks the row, to open or close the rows below it. Clicking the sign does not select the row and sends nothing to the server. `opendepth = "1"` in `gui.TableOptions` opens the rows of level 0 at first, so the rows of level 1 are visible. Without it, only the rows of level 0 (`Fruit` and `Vegetables`) are visible until the player opens them. The selected row in `ctx.form.files` is the position of the row in `cells`, so `Carrot` is `5` even when the rows above it are closed.

With `"indent"`, the rows are indented in the same way, but there is no sign and every row is always visible.

3.3.36 `gui.TableOptions`

Sets the appearance of the `gui.Table` that follows it. `opts` is a table with any of these options, all values are strings:

- `color`: Default text color (default `"#FFFFFF"`).
- `background`: Background color of the table (default `"#000000"`).
- `border`: `"true"` or `"false"`, whether to draw a border (default `"true"`).

- `highlight` : Background color of the selected row (default `"#466432"`).
- `highlight_text` : Text color of the selected row (default `"#FFFFFF"`).
- `opendepth` : How many levels of `"tree"` column rows are open at first. Rows deeper than this are collapsed (default `"0"`). See Example With Indent and Tree Columns.

Example

```
gui.VBox {
    gui.TableOptions {
        opts = {
            background = "#222222",
            highlight = "#335577",
            border = "false",
        },
    },

    gui.Table {
        w = 4, h = 2,
        name = "names",
        cells = { "Alice", "Bob", "Carol" },
        selected_idx = 2,
    },
}
```

Live Example

Pick a highlight color, select a row, and tick or untick the border.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Dropdown { name = "highlight", items = { "#335577", "#775533", "#337755" } },
        gui.Checkbox { name = "border", label = "Draw a border", selected = true },

        -- Style the table below.
        gui.TableOptions {
            opts = {
                background = "#222222",
                highlight = ctx.form.highlight or "#335577",
                border = tostring(ctx.form.border ~= false),
            },
        },
        gui.Table {
            w = 4, h = 3,
            name = "names",
            cells = { "Alice", "Bob", "Carol", "Dave" },
        },
    }
end)
```

3.3.37 `gui.Textarea`

A multi-line text box. Long lines are wrapped to fit the width, and a scrollbar is added if the text is too tall. `label`, if set, is shown above the box and `default` is the text it starts with. `ctx.form[name]` is the text in the box, including its newlines.

> **Without a `name`, a textarea is not editable**
>
> If you leave out `name`, the text cannot be edited and no box is drawn, which makes the textarea a label that wraps text and scrolls. This is easy to do by accident when you copy an example. It is also the way to show long text that wraps, because `gui.Label` does not wrap, see Unsupported Features. Because it wraps, you should set a `w` and `h` for this.

Example

```
gui.Textarea {
    w = 6, -- Optional
    h = 2, -- Optional
    name = "my_textarea", -- Optional
    label = "Hello world!", -- Optional

    -- Text the box starts with.
    default = "Hello world!", -- Optional
}
```

3.3.38 `gui.Textlist`

A list of text rows that scrolls if there are too many to fit. The player can select a row, and `ctx.form[name]` is the number of the selected row (the first row is `1`). `selected_idx` is the row that is selected at first.

`listelems` is the text of each row. To color a row, start it with `#RRGGBB` (six hexadecimal digits). To start a row with a literal `#`, write `##`. If `transparent` is `true`, no background is drawn behind the list.

Example

```
gui.Textlist {
    w = 4, -- Optional
    h = 2, -- Optional
    name = "my_textlist", -- Optional

    -- Text of each row. The second row is orange because it starts with a
    -- color, and the third starts with a literal #.
    listelems = { "Plain row", "#FF8800Orange row", "##Hash row", "Last row" },

    -- Row selected at first, the first row is 1.
    selected_idx = 3, -- Optional

    -- Do not draw a background. Default false.
    transparent = false, -- Optional
}
```

3.3.39 `gui.Tooltip`

Sets the text that is shown when the player hovers over another element, and optionally the colors of that tooltip. `gui_element_name` is the `name` of that element. Put the tooltip after the element it belongs to. It is invisible and does not take up any space.

If you only need to add some text, use the `tooltip` parameter on the element itself, see Tooltips.

Example

```
gui.VBox {
    gui.Button { name = "my_button", label = "Hover over me" },

    gui.Tooltip {
        -- Text to show.
        tooltip_text = "Hello world!",

        -- Background color of the tooltip.
        bgcolor = "#000000", -- Optional

        -- Text color of the tooltip.
        fontcolor = "#FFFF00", -- Optional

        -- Name of the element to attach to.
        gui_element_name = "my_button",
    },
}
```

Example

Hover over the button to see the tooltip. Click it and hover again: the text has changed.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Button {
            name = "my_button",
            label = "Click me, then hover",
            on_event = function(player, ctx)
                ctx.clicks = (ctx.clicks or 0) + 1
                return true -- redraw
            end,
        },
        gui.Tooltip {
            gui_element_name = "my_button",
            tooltip_text = ("Clicked %d times"):format(ctx.clicks or 0),
            bgcolor = "#000000",
            fontcolor = "#FFFF00",
        },
    }
end)
```

3.3.40 `gui.Vertlabel`

Shows text vertically, with one letter below the other, from top to bottom.

Example

```
gui.Vertlabel {
    label = "Hello world!",
}
```

3.4 Hypertext Markup

`gui.Hypertext` shows text that is written in a small markup language with tags that look like HTML tags. It can be used to make text bold, coloured, bigger, or centred, to add pictures, and to add clickable links.

The markup language is defined by Luanti, see Markup Language in the Luanti API documentation.

Warning: The markup language is currently unstable and may change. Use it with caution. Flow passes the text to Luanti without looking at it, so everything on this page is handled by the player's Luanti client, not by flow.

```
gui.Hypertext {
    w = 8, h = 3,
    text = "<center><big>Welcome!</big></center>" ..
        "This is <b>bold</b>, <i>italic</i>, and " ..
        "<style color='red'>red</style> text.",
}
```

What flow does not have: the markup can only be used in `gui.Hypertext`. Luanti can also show it in a tooltip (`hypertip[]`), but flow has no element for that, see Unsupported Features. Colors in the markup are written like colors everywhere else, see Colors: the client reads them with the same code, so `#RGB`, `#RRGGBB`, `#RGBA`, `#RRGGBBAA`, color names, and a name with an alpha value (`red#80`) all work. A color without an alpha value is fully opaque, and a color the client cannot read is ignored.

3.4.1 Syntax

Some tags enclose text: they open with `<tagname>` and close with `</tagname>`. Tags can have attributes, which are written in the opening tag as `key=value`. Attribute values should be quoted with either `"` or `'`. Inside a Lua string that is already in double quotes, use single quotes, as in `<style color='red'>`.

To write a literal `<`, `>`, or `\` in the text, put a backslash before it. In a quoted attribute value, put a backslash before a literal quote mark. In Lua code, each backslash has to be written twice (`"\\<"`), or you can use a long string (`[[\<]]`).

3.4.2 Basic Tags

`<style color=... font=... size=...>...</style>`

Changes the style of the text inside it.

- `color`: Text color.
- `size`: Text size, a whole number where `16` is the normal size (the size the player chose in their settings), so `32` is twice as big and `8` is half as big. `<big>` is `24` and `<bigger>` is `36`. The relative values from Styling are not supported: `*1.5` is ignored and `+2` is read as plain `2`.
- `font`: Text font, `mono` or `normal`.

`<global ...>`

Sets the style of the whole text. You only need to use it once. If you put several in the text, their changes are made in the order the tags appear.

Attributes that only apply to the whole text:

- `background` : Background of the text, a color or `none` .
- `margin` : Space around the edge of the text, in pixels.
- `valign` : Vertical alignment of the text: `top` , `middle` , or `bottom` .

Attributes that are inherited by everything inside the text:

- `color` : Default text color.
- `hovercolor` : Color of `<action>` links while the mouse is over them.
- `size` : Default text size, written like `size` in `<style>` .
- `font` : Default font, `mono` or `normal` .
- `halign` : Default horizontal alignment: `left` , `right` , `center` , or `justify` .

`<tag name=... color=... hovercolor=... font=... size=...>`

Defines a new tag, or changes the style of an existing one.

- `name` : The name of the tag to define or change.
- `color` : Text color.
- `hovercolor` : Text color while the mouse is over it (only for `action` tags).
- `size` : Text size, written like `size` in `<style>` .
- `font` : Text font, `mono` or `normal` .

3.4.3 Layout Tags

These tags are always available. You can add more with `<tag ...>`.

Tag	Effect
`<normal>...</normal>`	Normal size text
`<big>...</big>`	Big text
`<bigger>...</bigger>`	Bigger text
`<center>...</center>`	Centred text
`<left>...</left>`	Left-aligned text
`<right>...</right>`	Right-aligned text
`<justify>...</justify>`	Justified text
`<mono>...</mono>`	Monospaced font
`<b>...</b>`, `<i>...</i>`, `<u>...</u>`	Bold, italic, and underlined text

3.4.4 Links

`<action name=...>...</action>`

Makes the text inside it clickable. `name` is required. An optional `url` attribute is a web address to open when the text is clicked.

When the player clicks it, the form is sent to the server and the value of the `gui.Hypertext` element is `"action:"` followed by the action name. In flow, give the element a `name` and an `on_event` function, and read the value from `ctx.form`. This is experimental and may change.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Hypertext {
            w = 6, h = 1,
            name = "help",
            text = "Read the <action name='rules'>rules</action> or the " ..
                "<action name='faq'>FAQ</action>.",
            on_event = function(player, ctx)
                if ctx.form.help == "action:rules" then
                    ctx.reply = "No rules!"
                elseif ctx.form.help == "action:faq" then
                    ctx.reply = "Nobody has asked yet."
                end
                return true -- redraw to show the reply
            end,
        },
        gui.Label { label = ctx.reply or "Click a link." },
    }
end)
```

3.4.5 Pictures

`<img name=... float=... width=... height=...>`

Draws a texture that is in the client's media cache.

- `name` : The name of the texture (required).
- `float` : If present, the picture floats to the `left` or `right` and the text flows around it.
- `width` , `height` : Force a size instead of using the size of the texture. If you only give one of them, the shape of the texture is kept.

`<item name=... float=... width=... height=... rotate=... angle=...>`

Draws the inventory picture of an item.

- `name` : The item name (required), such as `unifieddyes:colorium` .
- `float` , `width` , `height` : Like for `<img>` .
- `rotate` : Make the item turn. Either `yes` or `X,Y,Z` , the rotation speeds around each axis as percentages of the standard speed (-1000 to 1000).
- `angle` : The angle the item is shown at, as `X,Y,Z` , the angles around each of the three axes.

`rotate` and `angle` only work if the player has the `inventory_items_animations` setting turned on.

4. Other features

4.1 Unsupported Features

Flow does not support everything that Luanti formspecs can do. This page lists what is missing, so you know before you look for it. Where there is a way to get a similar result, it is mentioned. Each formspec element links to its description in the Luanti API documentation.

4.1.1 Missing Elements

Formspec element	What it does	What to do instead
`hypertip[]`	A tooltip that shows formatted text (see Hypertext Markup).	Flow tooltips only show plain text with the `tooltip` parameter or `gui.Tooltip`. The markup language can only be used in `gui.Hypertext`.
`allow_close[]`	Stops the Escape key (and similar) from closing the form.	Nothing. Players can always close a flow form. You can run code when they do with `on_quit`, but you cannot stop it.
`listimages[]`	Pictures for the background of inventory slots.	`gui.Listcolors` only sets colors.
`label[x,y;w,h;text]`	A label that wraps long text inside a given size (an "area label").	`gui.Textarea` without a `name`, or `gui.Hypertext`. Because of this, the `halign` and `valign` styles do not do anything for `gui.Label`.
`tooltip[x,y;w,h;...]`	A tooltip for an area instead of an element.	The `tooltip` parameter of any element (flow uses the area of an element without a `name` for you). `gui.Tooltip` can only be attached to a named element.
`field[name;label;default]`	A small form with a single field and an automatic "Proceed" button (for example for signs).	Use a `gui.Field` and a `gui.Button`.

4.1.2 Missing Parameters

- **Content padding of scroll containers.** Luanti can work out the range of a scroll container's scrollbar for you, but flow's `gui.ScrollContainer` does not expose this, so you have to set the range yourself with `gui.ScrollbarOptions`. `gui.ScrollableVBox` already does this for you.

- **Node formspec inventory locations.** The `"current_node"` inventory location refers to the node whose formspec is open. A flow form cannot be the formspec of a node, so use `"nodemeta:X,Y,Z"` to show a node's inventory instead.

4.1.3 Things Flow Does for You

These formspec elements exist, but flow generates them itself and you cannot write them:

- `size[]` : The size of the form is worked out from the elements in it.
- `position[]` , `anchor[]` , and `padding[]` : Use the `window_position` , `window_anchor` , and `window_padding` options of the root element, see Window Position.
- `no_prepend[]` and `bgcolor[]` : Use the `no_prepend` option and the `bgcolor` , `fbgcolor` , and `bg_fullscreen` options of the root element.
- `real_coordinates[]` : Flow always uses the newer coordinate system, where there is no hidden padding or spacing between elements.
- `container_end[]` and `scroll_container_end[]` : Elements inside `gui.Container` and `gui.ScrollContainer` are written inside the table.
- `formspec_version[]` : Flow always uses formspec version 7.

4.1.4 Newer Luanti Features

Because flow always uses formspec version 7 (Luanti 5.8), features that were added in newer versions may not work, even if the missing element or parameter is not listed above. This includes the area label (version 9), floating-point animation frames in `gui.Model` (version 10), and the `halign` and `valign` styles of fields and text areas (version 11), and the `editable` style of checkboxes, dropdowns, and fields (version 12, the only addition in that version). Formspec versions are described in the "Formspec Version History" section of the Luanti API documentation.

Flow does not check the formspec version when it writes style properties. A property such as `halign` , `valign` , or `editable` is put in the `style[]` or `style_type[]` element exactly as you gave it, and is not removed or changed.

The version that flow declares (7) does not stop a newer feature from working. In the current Luanti client source, the client does not compare the form's version with the version a feature was added in. It only asks whether the client itself knows the feature. So `halign` and `valign` work for players on Luanti 5.17 or newer, `editable` works on Luanti 5.18 or newer, and the area label (an extra parameter of `label[]`) is accepted by clients that know it, all from a form that says version 7.

What happens on a client that is too old depends on the feature. In the current source, a style property that the client does not know is skipped and the other properties of the same `style[]` still apply. An element with more parameters than the client knows is not drawn at all, and the client logs an error. Clients older than the current source were not checked.

Formspec Versions and Luanti Releases

Formspec version	Luanti release	Notable additions
2	5.1.0	Real coordinates always on, `background9[]`
3	5.2.0	Draw order follows definition order, `scroll_container[]`
4	5.4.0	Dropdown index events
5	5.5.0	`padding[]`
6	5.6.0	Nine-slice images, `animated_image[]`
7	5.8.0	Focused button state, `field_enter_after_edit[]` (the version flow uses)
8	5.10.0	Scroll container content padding
9	5.12.0	`allow_close[]`, area `label[]`
10	5.13.0	Floating-point `model[]` frames
11	5.17.0	`hypertip[]`, text alignment styles
12	5.18.0	`editable` style

4.1.5 Scrollbar Textures Do Not Work

`scrollbar_bg`, `slider`, `arrow_up`, and `arrow_down` should not be used. Flow accepts them, but Luanti has no such parameters, and on Luanti 5.8 and newer a scrollbar that sets any of them is not shown at all.

Flow (through formspec_ast) writes the four textures as one comma-separated sixth parameter after `value`, leaving the ones you did not set empty. Luanti's `scrollbar[]` takes exactly five parameters. It only tolerates extra

parameters when the form's formspec version is newer than the highest version the client supports, and flow always uses version 7:

Luanti version	Highest formspec version	Result
5.6 and 5.7	6	The extra parameter is ignored and the scrollbar is shown with its normal look.
5.8 and newer	7 or higher	The scrollbar is rejected and not shown, and an error is logged.

The rejection was confirmed in-game with a `gui.Scrollbar` that set all four textures and one that set only `slider`. A scrollbar without textures next to them was shown normally.

Sources (Luanti source code):

- `GUIFormSpecMenu::parseScrollBar` requires between 5 and 5 parameters, and nothing in `guiFormSpecMenu.cpp` reads the textures: 5.8.0, 5.7.0, master.
- `GUIFormSpecMenu::precheckElement` in the same file accepts extra parameters only when `m_formspec_version > FORMSPEC_API_VERSION`, and otherwise logs `Invalid <element> element(<count>)` and skips the element.
- `FORMSPEC_API_VERSION` is 6 in 5.7.0, 7 in 5.8.0, and 12 in master.
- The formspec documentation lists `scrollbar[]` with five parameters: position, size, orientation, name, and value.

4.2 Security

A player's client can send anything to the server. A normal client only sends values that the form could produce, but a modified one can send other names, other values, and several actions at once. This page lists what flow checks before it stores a value or calls one of your functions, and what it leaves to you.

4.2.1 What Flow Checks

- **The form.** Flow only handles input for the form that the player has open right now. Every time a form is shown with `form:show()` it gets a new internal name, so input that is meant for an old or made-up form is ignored.
- **The elements.** Flow only stores values for named inputs that were in the form it drew, and only calls `on_event` and `on_key_enter` functions of elements that were in it. Anything else the client sends is ignored. Hidden elements are not part of the form, see Hiding Elements.
- **Size.** A value longer than 60,000 bytes is ignored and a warning is logged.
- **Text.** Control characters other than tabs and newlines are removed from the text of `gui.Field`, `gui.Pwdfield`, and `gui.Textarea`, and from the value of a `gui.Hypertext` action. The same goes for the control characters from U+0080 to U+009F. Newlines are also removed from `gui.Field`, where a player can only enter one by pasting it.
- **Checkboxes.** The value is always `true` or `false`.
- **Dropdowns.** The value must be one of the items. With `index_event = true` it must be a number from `1` to the number of items. On clients that support it, flow always asks for the item number instead of the text, so the client never gets to choose the text.
- **Tabs, tables, and text lists.** The selected tab or row must be a number from `1` to the number of tabs or rows. Decimals are rounded down.
- **Buttons.** Flow calls the `on_event` function of at most one button per submission, and not at the same time as an `on_key_enter` function. Buttons never store a value.

When a value is rejected, `ctx.form` keeps its previous value. The other values in the same submission are still used.

4.2.2 What You Still Need to Check

- **Who is allowed to do something.** Flow can tell that a button exists in the form, but not whether the player really clicked it. A modified client can trigger the `on_event` function of any element that is currently in the form. Check privileges, ownership, distances and so on inside the function, the same as you would in a chat command.
- **What is in the value.** Flow does not limit text to a length (apart from 60,000 bytes), and it does not check scrollbar positions against the range of the scrollbar. Check values that matter before you use them, for example before putting text into a node's metadata.
- **Inventories.** The slots of a `gui.List` are handled by Luanti, flow does not check or limit what is moved. Use the inventory callbacks that Luanti provides.

- **Hypertext.** Flow does not look at the markup that you give `gui.Hypertext`, see Hypertext Markup. Escape text from players with `core.hypertext_escape()`.

Everything in `ctx` except `ctx.form` is only changed by your code, so the client cannot change it.

4.2.3 Text in Elements

Text that you put in an element, such as a label, is escaped when flow generates the formspec, so it cannot add or change other elements.

4.3 Styling Forms

Styles change how elements look: their colors, images, fonts, and so on. A style is a set of properties such as `bgcolor = "red"`, and which properties exist depends on the type of element. All of them are listed in the property reference below.

You can style a single element with the `style` field (the inline syntax), or style many elements at once with the `gui.Style` and `gui.StyleType` elements (see separate style elements).

Which one to use:

You want to	Use
Style one element, or a few that share a table	The inline `style` field
Style one element by its `name`, or support old versions of flow	`gui.Style`
Style every element of a type (every button, say) after this point	`gui.StyleType`
Change the size of an element (`font_size`, list `size` and `spacing`)	The inline `style` field only, see Styles That Change Size

Two things show up in the examples below. A `sel` entry (inline) or a selector (separate elements) limits a style to some states of the element, such as `hovered`. Within one entry, states separated by a comma apply when any of them is active and states joined with `+` apply only when all of them are active. In a `sel` entry, `$` stands for the element itself. These are explained in full under Inline Syntax and Selectors and States.

4.3.1 Inline Syntax

Inline styles were added in May 2024, so older versions of flow do not support this syntax, and the sub-style syntax (the `sel` entries) may change in the future. Only use inline styles for things that are not essential, so that your form does not look broken without them, or use separate style elements if you need to support older versions.

You can add inline styles to elements with the `style` field:

```
gui.Button {
    label = "Test",
    style = {
        bgcolor = "red",

        -- You can style specific states of elements:
        { sel = "$hovered", bgcolor = "green" },

        -- Or a combination of states:
        { sel = "$hovered, $pressed", bgcolor = "blue" },
        { sel = "$hovered+pressed", bgcolor = "white" },
    },
}
```

In a `sel` entry, `$` stands for the element itself, so `"$hovered+pressed"` is the same as the selector `"button:hovered+pressed"` for an unnamed button, or `"my_name:hovered+pressed"` for an element named `my_name`. Every comma-separated entry needs its own `$` (an entry without one is skipped and flow logs a warning).

The properties directly inside `style` always apply. A `sel` entry only applies while the element is in one of the listed states. Separate states with a comma to apply the style when *any* of them is active (`"$hovered, $pressed"`), or join them with `+` to apply it only when *all* of them are active at once (`"$hovered+pressed"`).

If you need to style multiple elements, you can reuse the `style` table:

```
local my_style = { bgcolor = "red", { sel = "$hovered", bgcolor = "green" } }

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Button { label = "Styled button", style = my_style },
        gui.Button { label = "Unstyled button" },
        gui.Button { label = "Second styled button", style = my_style },
    }
end)
```

Styling an unnamed element can undo other styles

Luanti can only style unnamed elements by their type, so flow applies the style to every element of that type and then resets those properties to their defaults straight after the element. This also undoes any earlier `gui.StyleType` for the same properties, for every element of that type that comes after. Give the element a `name` to avoid this.

In this form, button A is blue because of the `gui.StyleType`, button B is red, and button C is not blue: the style on B reset `bgcolor` for every button after it. Button D has a `name`, so its style is applied to it alone and nothing is reset:

```
gui.VBox {
    gui.StyleType { selectors = { "button" }, props = { bgcolor = "blue" } },

    gui.Button { label = "A" },
    gui.Button { label = "B", style = { bgcolor = "red" } },
    gui.Button { label = "C" },
    gui.Button { name = "d", label = "D", style = { bgcolor = "red" } },
}
```

Property values are normally strings, but flow also accepts numbers and booleans where a property needs one. Setting a property to an empty string (`bgimg = ""`) resets it to its default value.

4.3.2 Separate Style Elements

Alternatively, you can use the `gui.Style` and `gui.StyleType` elements if you need to style a large group of elements or need to support older versions of flow. This form has a second button that is left alone so you can compare:

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Style {
            selectors = { "btn1" },
            props = {
                bgcolor = "red",
                border = false,
            },
        },

        gui.Button { name = "btn1", label = "Styled by name" },
        gui.Button { name = "btn2", label = "Not styled" },
    }
end)
```

The `Style` and `StyleType` elements are invisible and won't affect padding. They only affect the elements that come after them, so put them before the elements you want to style.

4.3.3 Styles That Change Size

Flow works out the size of an element before it writes any styles, so it has to see a style that changes the size. It only does that in some cases:

- `gui.Style` and `gui.StyleType` do not support styles that change the element size. Flow only sees these properties in the inline syntax, so use that for them.
- `font_size` changes the size of `gui.Label`s, buttons, and the default text of fields only if it starts with `*` (such as `"*2"`) and is set with the inline syntax. Other values still change the text but not the size of the element.

- The labels above fields and checkboxes ignore `font_size`, and a `gui.Label` with a fixed `h` is positioned as if `font_size` was not set. That behavior is deprecated and flow logs a warning when it detects it, so leave `h` out of labels that use `font_size`.
- The `size` and `spacing` styles of `gui.List` (added in March 2025) count too, but only whole numbers such as `"2,2"` or `"0,0"` and only in the inline syntax.

4.3.4 Selectors and States

`selectors` is a list that says which elements the style applies to:

- `gui.Style` selects elements by their `name`, so `{"btn1"}` styles the element with `name = "btn1"`.
- `gui.StyleType` selects elements by their type, so `{"button"}` styles every button that comes after it. The type is the element's name in lowercase with underscores between words, for example `item_image_button` for `gui.ItemImageButton`. Some types also style other types, see element types.
- `gui.StyleType` also accepts `"*"`, which selects every element.

A selector can end with a colon and one or more states to only style the element while it is in that state, such as `"btn1:hovered"` or `"button:hovered+pressed"` (`+` means all of the states have to be active).

In this form, the text turns yellow on every button, and a button turns blue while the mouse is over it. The label is not a button, so it is not affected:

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.StyleType {
            selectors = { "button", "image_button" },
            props = { textcolor = "yellow" },
        },
        gui.StyleType {
            selectors = { "button:hovered" },
            props = { bgcolor = "#336699" },
        },

        gui.Button { label = "Hover over me" },
        gui.Button { label = "Me too" },
        gui.Label { label = "I am not a button." },
    }
end)
```

States

Every element has the `default` state, which is the same as not giving any state. Buttons (`gui.Button` , `gui.ButtonExit` , `gui.ImageButton` , `gui.ItemImageButton`) also have these states:

State	Active when
`focused`	The button has keyboard focus
`hovered`	The mouse is over the button
`pressed`	The button is being pressed

Element Types

These are the types that can be styled. A type that "inherits" from another also uses the styles set for the other type, for example a style set on `image` also applies to `animated_image` .

Type	Element	Inherits from
`animated_image`	`gui.AnimatedImage`	`image`
`box`	`gui.Box`	
`button`	`gui.Button`	
`button_url`	`gui.ButtonURL`	Not valid, style by `name`
`button_exit`	`gui.ButtonExit`	`button`
`checkbox`	`gui.Checkbox`	
`dropdown`	`gui.Dropdown`	
`field`	`gui.Field`	
`image`	`gui.Image`	
`image_button`	`gui.ImageButton`	
`image_button_exit`	`gui.ImageButtonExit`	Not valid, style by `name`
`item_image`	`gui.ItemImage`	Only supports `noclip`
`item_image_button`	`gui.ItemImageButton`	
`label`	`gui.Label`	
`list`	`gui.List`	
`model`	`gui.Model`	
`pwdfield`	`gui.Pwdfield`	`field`
`scrollbar`	`gui.Scrollbar`	
`tabheader`	`gui.Tabheader`	
`table`	`gui.Table`	
`textarea`	`gui.Textarea`	
`textlist`	`gui.Textlist`	
`vertlabel`	`gui.Vertlabel`	`label`

Luanti also lists `hypertip`, but flow has no element for it, see Unsupported Features. The `item_image` type only supports `noclip` (see Other Elements). Types marked "Not valid" are not in Luanti's list of types, so a `gui.StyleType` for them does nothing: give those elements a `name` and style them with `gui.Style` or the inline syntax.

4.3.5 Property Reference

Colors are described in Common Aspects, and textures are image file names. A "2D vector" is written as a string such as `"1,2"`. Properties that only list a type of element are not valid on other elements, and an unknown property is ignored.

All Elements

Property	Description
`noclip`	Boolean. Set to `true` to let the element be drawn outside the area of the form. Default `false`.

Text Properties

These properties change how text looks. `font` and `font_size` are valid on buttons, labels, fields, tables, and text lists. `textcolor` is valid on buttons, labels, fields, and tab headers, but not on tables and text lists. The sections below show which applies where.

Property	Description
`font`	Font type, a comma-separated list. Use at most one of `normal` (the default font) and `mono` (a font where every letter is the same width), and any of `bold` and `italic`. For example `"mono,bold"`. If you only give `bold` or `italic`, `normal` is used. Default `normal`.
`font_size`	Text size. `"16"` sets an absolute size, `"+2"` and `"-2"` make the text 2 points larger or smaller than the default, and `"*1.5"` multiplies the default size by 1.5 (like `em` in CSS). The default is the size the player has chosen in their settings.
`textcolor`	Color of the text. Default white.

Which `font_size` values change the size of an element is described in Styles That Change Size.

Buttons

For `gui.Button`, `gui.ButtonExit`, `gui.ImageButton`, and `gui.ItemImageButton`. Buttons also support `font`, `font_size`, and `textcolor`.

Property	Description
`alpha`	Boolean. Whether to draw the transparent parts of `bgimg`. Default `true`.
`bgcolor`	Color that tints the button.
`bgimg`	Texture to use as the background of the button. Default none.
`bgimg_middle`	Draws `bgimg` as a nine-slice. Written as a string in one of the formats `"x"`, `"x,y"`, or `"x,y,x2,y2"`, which have the same meaning as `middle_x`, `middle_y`, `middle_x2`, and `middle_y2`. Also adds padding around the content of the button.
`border`	Boolean. Whether to draw the bevelled button border. Set to `false` to hide it. Default `true`.
`content_offset`	2D vector that moves the content (text and image) of the button without resizing the button.
`padding`	Space between the edges of the button and its content, as a rectangle written in the same formats as `bgimg_middle`. This value is relative to `bgimg_middle`. It is not the same as the `padding` parameter of elements, which adds space outside the element and is handled by flow, see Padding.
`sound`	The name of a sound to play when the button is triggered, such as a sound file name from a mod's `sounds` folder without the extension. It is played at normal volume and takes no other options.

`gui.ImageButton` has some more properties for the picture drawn on top of the button:

Property	Description
`fgimg`	The picture to draw. Default none. The `texture_name` and `pressed_texture_name` you give to the element take precedence over this.
`fgimg_middle`	Draws `fgimg` as a nine-slice, like `bgimg_middle`.

Some older properties still work but are deprecated. Use states instead: `bgcolor_hovered`, `bgcolor_pressed`, `bgimg_hovered`, `bgimg_pressed`, `fgimg_hovered`, and `fgimg_pressed`. They work like `bgcolor` and `bgimg` but only apply while the button is hovered or pressed.

Fields and Text Areas

For `gui.Field`, `gui.Pwdfield`, and `gui.Textarea`. They also support `font`, `font_size`, and `textcolor`.

Property	Description
`border`	Boolean. Set to `false` to hide the background and border of the text box. Default `true`.
`halign`	Horizontal alignment of the text: `left` (default), `center`, or `right`. Added in formspec version 11 (Luanti 5.17, see the version table). The form's version does not matter, so it works for any player on Luanti 5.17 or newer even though flow declares version 7, and a client that does not know it skips the property, see Newer Luanti Features.
`valign`	Vertical alignment of the text: `top` (default), `center`, or `bottom`. Added in formspec version 11 (Luanti 5.17, see the version table). The form's version does not matter, so it works for any player on Luanti 5.17 or newer even though flow declares version 7, and a client that does not know it skips the property, see Newer Luanti Features. Only has an effect when all of the text fits inside the element. If the text is too long (and a `gui.Textarea` gets a scrollbar), it is always aligned to the top so that no text is cut off. It also does not work in `gui.Field`, but `halign` does.

Labels

For `gui.Label` and `gui.Vertlabel`: `font`, `font_size`, and `textcolor`.

Luanti also has `halign` and `valign` for labels, but they only apply to the area label, which flow does not have, so they do nothing here (see Unsupported Features).

Lists

For `gui.List`:

Property	Description
`size`	2D vector. The size of each slot. Default `"1,1"`.
`spacing`	2D vector. The space between slots. Default `"0.25,0.25"`.

Flow uses these values to work out how big a list is, see Styles That Change Size for what it understands.

Boxes

For `gui.Box`. These properties only take effect when the box has no `color` of its own. The `colors`, `bordercolors`, and `borderwidths` properties can have one, two, or four values separated by commas (like CSS borders):

- One value, such as `"#FF0"`: used for all corners or borders.
- Two values, such as `"red,#FFAAFF"`: for the colors, the top-left and bottom-right corners, then the top-right and bottom-left corners. For the widths, the top and bottom borders, then the left and right borders.
- Four values, such as `"blue,#A0F,green,#FFFA"`: starting with the top-left corner (or the top border) and going clockwise.

Property	Description
`colors`	Color of the corners of the box. Default `black`.
`bordercolors`	Color of the borders of the box. Default `black`.
`borderwidths`	Width of the borders in pixels, as a whole number. A negative width puts the border inside the box, a positive width puts it outside the box, and `0` draws no border. Default `0`.

Models

For `gui.Model`:

Property	Description
`bgcolor`	Background color of the model area.

Read-Only Elements (`editable`)

Luanti 5.18 (formspec version 12) added an `editable` property for `gui.Checkbox` , `gui.Dropdown` , `gui.Field` , `gui.Pwdfield` , and `gui.Textarea` . Setting it to `false` makes the element read-only (default `true`). Flow does nothing special for it and writes the property unchanged. The client does not check the form's version for it, so it works for players on Luanti 5.18 or newer even though flow declares version 7, and it may be ignored for other players. Do not rely on it alone to stop changes: also ignore changes to the element in your own code. See also Newer Luanti Features.

Other Elements

Element	Properties
`gui.Checkbox`	`sound` : A sound to play when the box is ticked or unticked.
`gui.Dropdown`	`sound` : A sound to play when a different item is selected.
`gui.Tabheader`	`sound` : A sound to play when a different tab is selected. `textcolor` : Color of the tab text, default white.
`gui.Table` , `gui.Textlist`	`font` and `font_size` . The colors of a table are set with `gui.TableOptions` .
`gui.Image` , `gui.AnimatedImage` , `gui.ItemImage` , `gui.Scrollbar`	Only `noclip` .

4.4 Padding, Spacing, and Backgrounds

Most elements can have a `padding` value, which will add the specified amount of padding around the element, measured in slot units. The "root" element of the form (the one returned by the build function) has a default padding of 0.3, everything else has a default padding of 0.

`padding` is ignored on these elements:

- `gui.Flow`, `gui.ScrollableVBox`, and `gui.PaginatedVBox`. To pad one, put it inside a `gui.VBox` (or another layouting element) and set `padding` on that instead.
- Elements inside a `gui.Container`, which are placed at their `x` and `y` and are not padded.
- Invisible elements such as `gui.Style`, which take up no space.

There is also a deprecated `gui.Padding` element that wraps a single element and pads it. Use the `padding` parameter instead.

`HBox` and `VBox` have a `spacing` field which specifies how much spacing there is between elements inside the box. Its default value is described in `gui.VBox`.

Box elements (`gui.HBox` and `gui.VBox`) can optionally have `bgimg` and `bgimg_middle` parameters that specify a background for the box. `bgimg` is the name of a texture, which is stretched to fill the box. If `bgimg_middle` is set as well, the texture is drawn as a nine-slice whose corners are `bgimg_middle` pixels big, so one small texture can make a background of any size. The background will be drawn behind any padding that the box has.

The examples below use a small 16x16 texture, a white border around a blue square. It is made with texture modifiers, so no file is needed. Because the border is only 2 pixels wide, you can see how it is stretched.

Example with only `bgimg`. The whole texture is stretched, so the border gets thick and blurry:

```
gui.VBox {
    -- Stretch the texture over the whole box
    bgimg = "[fill:16x16:white^[fill:12x12:2,2:#336699",

    gui.Button { label = "Button 1" },
    gui.Button { label = "Button 2" },
}
```

Example with `bgimg_middle` as well. The corners and edges keep their 2 pixel border, and only the middle is stretched:

```
gui.VBox {
    padding = 0.5,
    spacing = 0.1,

    -- Draw the texture as a nine-slice
    bgimg = "[fill:16x16:white^[fill:12x12:2,2:#336699",
    bgimg_middle = 2,

    gui.Button { label = "Button 1" },
    gui.Button { label = "Button 2" },
}
```

The padding around the VBox is 0.5 and the spacing between the buttons inside it is 0.1.

4.5 Tooltips

A tooltip is a small box of text that appears when the player hovers over an element.

You can add tooltips to elements using the `tooltip` field:

```
gui.Image {
    w = 2, h = 2,

    -- A red square, made with a texture modifier so that this example does
    -- not need any files.
    texture_name = "[fill:16x16:red",
    tooltip = "A red square",
}
```

Hover over the square to see its tooltip. Here is a version with more elements to hover over:

```
gui.VBox {
    gui.Button { label = "Hover over me", tooltip = "I am a button tooltip!" },
    gui.Label { label = "Hover over me too", tooltip = "Labels work as well." },
}
```

This works on elements without a `name` as well. In that case the tooltip covers the area of the element, and the tooltip of a named element that overlaps it takes priority.

If the areas of two unnamed elements overlap and both have a tooltip (for example in a `gui.Stack`, or a box with a tooltip that contains an element with its own), the one that comes first in the form wins. Flow puts each tooltip directly before its element, and Luanti shows the first area tooltip that contains the mouse pointer, so the tooltip of a box beats the tooltips of the elements inside it.

There is also a `gui.Tooltip` element which lets you change the background and text color of the tooltip. It refers to the element it belongs to with `gui_element_name`, which must be the `name` of that element, and it should come after that element. As with `gui.Style`, it is invisible and won't affect padding.

```
gui.VBox {
    gui.Button { name = "my_button", label = "Hover over me" },
    gui.Tooltip {
        gui_element_name = "my_button",
        tooltip_text = "Hello!",
        bgcolor = "#000000",
        fontcolor = "#FFFF00",
    },
}
```

Tooltips are rebuilt every time the form is drawn, so their text can change. Click the button below, then hover over it again:

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Button {
            name = "my_button",
            label = "Click me, then hover",
            on_event = function(player, ctx)
                ctx.clicks = (ctx.clicks or 0) + 1
                return true
            end,
        },
        gui.Tooltip {
            gui_element_name = "my_button",
            tooltip_text = ("Clicked %d times"):format(ctx.clicks or 0),
            bgcolor = "#000000",
            fontcolor = "#FFFF00",
        },
    }
end)
```

Flow cannot make tooltips that show formatted text (`hypertip[]`) or tooltips for an arbitrary area instead of an element, see Unsupported Features.

4.6 Hiding Elements

Elements inside boxes can have `visible = false` set to hide them from the player. Elements hidden this way will still take up space like with `visibility: hidden;` in CSS.

4.6.1 Example

Untick the checkbox to hide the middle button: it disappears, but the gap it leaves stays.

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Checkbox { name = "show", label = "Show the middle button", selected = true },
        gui.Button { label = "First button" },
        gui.Button { label = "Middle button", visible = ctx.form.show ~= false },
        gui.Button { label = "Last button" },
    }
end)
```

4.6.2 Where It Works

- `visible = false` works on elements inside `gui.HBox`, `gui.VBox`, `gui.Stack`, `gui.ScrollableVBox`, and `gui.PaginatedVBox`. In a `gui.Stack`, the hidden element still counts towards the size of the stack.
- It does nothing on elements placed directly inside a `gui.Container` or `gui.ScrollContainer` (the ones you position yourself with `x` and `y`): they are still drawn. See Manual Positioning.
- It does nothing on the root element of the form, which is always shown.

4.6.3 What Happens to Hidden Elements

Flow takes a hidden element out of the form after it has worked out the layout, and before it handles anything else. The player's client never receives it, so:

- A hidden button cannot be clicked or focused, and its `on_event` function is never called. The same goes for every other hidden element.
- Its `style` and `tooltip` are not sent either.
- A hidden input with a `name` is not registered, so flow does not set or update `ctx.form[name]` while it is hidden. A value that is already in `ctx.form` is kept and is applied again when the element is shown.

Type something in the field of this example, then untick the checkbox. The field disappears, but the label still shows what you typed, and the text is back in the field when you tick the checkbox again:

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Checkbox { name = "show_note", label = "Show the note field", selected = true },
        gui.Field { name = "note", label = "Note", visible = ctx.form.show_note ~= false },
        gui.Label { label = "Stored note: " .. (ctx.form.note or "") },
    }
end)
```

While the field is hidden, flow ignores a `note` value sent by the client, because the field is not registered.

4.6.4 Alternatives

If you don't want hidden elements to take up any space, see the documentation for `gui.Nil`. `gui.Nil` is an invisible element (a tooltip) that is hidden with `visible = false`, so it is skipped when the layout is worked out. See Invisible Elements for the others. An invisible element that is itself given `visible = false` is removed from the form in the same way, it just never had a size.

4.7 Using a Form as an Inventory

A form can be set as the player inventory, which is the screen the player sees when they open their inventory. Flow internally generates the formspec (the text Luanti uses to describe a menu) and passes it to `player:set_inventory_formspec()`. This will completely replace your inventory and isn't compatible with inventory mods like sfinv (the inventory in Minetest Game, which lets mods add tabs). To put a flow form inside an sfinv-style inventory instead, see Rendering to a Formspec.

> Consider using Sway instead if you want to use flow as an inventory replacement while still having some way for other mods to extend the inventory.

```
local gui = flow.widgets

local example_inventory = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = ctx.title or "Inventory goes here!" },
        gui.Label { label = "Button clicks: " .. (ctx.clicks or 0) },
        gui.Button {
            label = "Click me",
            on_event = function(player, ctx)
                ctx.clicks = (ctx.clicks or 0) + 1
                return true -- redraw
            end,
        },
    }
end)
core.register_on_joinplayer(function(player)
    example_inventory:set_as_inventory_for(player)
end)

-- /inv_title <text> changes the title, see below.
core.register_chatcommand("inv_title", {
    params = "<text>",
    description = "Change the title of your inventory form",
    func = function(name, param)
        local player = core.get_player_by_name(name)
        if player then
            example_inventory:set_as_inventory_for(player, { title = param })
        end
    end,
})

-- /inv_off removes the form, see below.
core.register_chatcommand("inv_off", {
    description = "Remove the custom inventory form",
    func = function(name)
        local player = core.get_player_by_name(name)
        if player then
            example_inventory:unset_as_inventory_for(player)
        end
    end,
})
```

Open your inventory and click the button: the counter goes up, and the form is redrawn with the new context.

Like with the `show_hud` function (see HUDs), `update*` functions don't do anything, so to update it, call `example_inventory:set_as_inventory_for(player, new_ctx)` again with the new context. If the context is not provided, it will reuse the existing context of the same form. This is different from HUDs, which never keep a context. Passing a new context replaces the existing one, so values that were stored in the old one are gone. See Context Lifetime by Mode.

The `/inv_title` chat command above does this. Keep your inventory open while you run it and the title changes straight away. The command passes a new context, which replaces the old one, so the click counter goes back to 0.

If the player has the inventory open when you do this, it is redrawn straight away: Luanti compares the new formspec with the one on screen and rebuilds the screen if they differ. As with any redraw, text that the player has typed into a field but not yet sent is lost. If the new formspec is identical to the old one, nothing is sent.

Flow does the same when the player does something in the inventory that needs a redraw (an `on_event` function returning `true`, for example). Other changes to `ctx.form` that do not need a redraw are applied when the player closes the inventory, so they show up the next time it is opened.

While the form will of course be cleared when the player leaves, if you'd like to unset the inventory manually, call `example_inventory:unset_as_inventory_for(player)`, analogue to `close_hud` (see HUDs).

For example, after the `/inv_off` chat command above, the inventory key shows the default inventory (or nothing, depending on your game) instead of the form.

This will set the inventory formspec string to `""` and stop flow from processing inventory formspec input.

4.8 Using a Form as a HUD

A form can be shown as a HUD (text and pictures drawn over the game screen) instead of a menu. A HUD cannot be clicked or typed into, so it is only useful for showing information such as a score, a timer, or a status display.

Flow uses the hud_fs mod to do this. It is an optional dependency, so it must be installed separately. hud_fs only draws some of the formspec elements, and ignores the others, see its page for the list.

4.8.1 Showing and Closing a HUD

```
local gui = flow.widgets
local player = core.get_player_by_name("current_player")

local my_gui = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Put the HUD in the top-right corner.
        window_position = { x = 1, y = 0 },
        window_anchor = { x = 1, y = 0 },

        gui.Label { label = ctx.text },
    }
end)

-- Draw the form as a HUD, or redraw it if it is already shown.
my_gui:show_hud(player, { text = "I am a HUD!" })

-- Remove the HUD again after 5 seconds.
core.after(5, function()
    my_gui:close_hud(player)
end)
```

- `show_hud(player, ctx)` draws the form as a HUD, or redraws it if it is already shown. Without hud_fs it stops with an error. `ctx` is optional.
- `close_hud(player)` removes the HUD.

Flow does not keep a context for HUDs. If you leave out `ctx`, `show_hud` uses a new empty table every time it is called, so keep your own table and pass it each time if the form needs to remember anything. This is different from inventory forms, which reuse their context. The table in Context Lifetime by Mode compares all the ways of showing a form.

4.8.2 Updating a HUD

`my_gui:update(player)` and `my_gui:update_where(func)` only affect forms shown with `my_gui:show(player)`, so they do nothing to a HUD. Call `show_hud` again to update it.

4.8.3 Example

```
local gui = flow.widgets

local hud = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Put the HUD in the top-right corner.
        window_position = { x = 1, y = 0 },
        window_anchor = { x = 1, y = 0 },

        gui.Label { label = "Score: " .. ctx.score },
    }
end)

local contexts = {}

local function set_score(player, score)
    local name = player:get_player_name()
    contexts[name] = contexts[name] or {}
    local ctx = contexts[name]
    ctx.score = score

    if not ctx.hidden then
        -- Pass the same context every time.
        hud:show_hud(player, ctx)
    end
end

core.register_on_joinplayer(function(player)
    set_score(player, 0)
end)

core.register_on_leaveplayer(function(player)
    contexts[player:get_player_name()] = nil
end)

-- Add a point every second so the HUD visibly changes.
local timer = 0
core.register_globalstep(function(dtime)
    timer = timer + dtime
    if timer < 1 then return end
    timer = 0

    for _, player in ipairs(core.get_connected_players()) do
        local ctx = contexts[player:get_player_name()]
        if ctx then
            set_score(player, ctx.score + 1)
        end
    end
end)

-- /hud off removes the HUD, /hud on brings it back.
core.register_chatcommand("hud", {
    params = "<on|off>",
    description = "Show or hide the score HUD",
    func = function(name, param)
        local player = core.get_player_by_name(name)
        local ctx = contexts[name]
        if not player or not ctx then return end
```

```
        if param == "off" then
            ctx.hidden = true
            hud:close_hud(player)
        else
            ctx.hidden = nil
            hud:show_hud(player, ctx)
        end
    end,
})
```

The score in the top-right corner goes up by one every second. Run `/hud off` to remove it with `close_hud` and `/hud on` to show it again with `show_hud`.

4.8.4 Things to Know

- **No input.** `on_event` and `on_key_enter` functions are never called, as the player cannot use a HUD.
- **Position and size.** Use `window_position`, `window_anchor`, and `window_padding` on the root element to place the HUD, see Experimental Features.
- **Scrolling.** Flow does not know the player's formspec version when it draws a HUD, so it treats the player as an old client. `gui.ScrollableVBox` becomes a `gui.PaginatedVBox`, whose page buttons do nothing in a HUD. Avoid both of them.

4.9 Manual Positioning

4.9.1 Dynamic Element Types

A `gui.Label{...}` call just returns a table with `type = "label"` added to it, so you can write `{type = "label", label = "Hello world!"}` instead of `gui.Label{label="Hello world!"}`. This is what you need inside `gui.Container` (see below), and it is also useful if you want to choose the element type from a variable. The `type` is the name of the `gui` function in lowercase with underscores between words, for example `"item_image_button"` for `gui.ItemImageButton`. The element types table in Styling lists many of them.

HBoxes and VBoxes can be created this way as well (with `type = "hbox"` and `type = "vbox"`) and work anywhere `gui.HBox` and `gui.VBox` do. However, `gui.ScrollableVBox`, `gui.PaginatedVBox`, `gui.Spacer`, `gui.Flow`, and `gui.Nil` are functions that build other elements instead of element types, so they cannot be created with a `type` table. The flow mod does not recognize those type names, and showing the form fails with an `Unknown element` error.

An example of this is in `example.lua` in the flow repository.

4.9.2 Manual Positioning of Elements

You can use `gui.Container` elements to contain manually positioned elements. Every element inside a container has an `x` and `y`, which is the distance of its top-left corner from the top-left corner of the container, in the same units as `w` and `h`. Containers can be nested, in which case the offsets add up. If you do not give a `w` and `h`, flow uses the smallest size that fits all the elements inside.

Container: Basic Example

```
gui.Container {
    -- You may embed most plain element tables (a `type`, an `x` and `y`,
    -- and the same parameters as the matching gui function) in a container
    padding = 0,
    { type = "box", x = 0, y = 0, w = 1, h = 1, color = "red" },
    { type = "box", x = 0.3, y = 0.3, w = 1, h = 1, color = "green" },
    { type = "box", x = 0.6, y = 0.6, w = 1, h = 1, color = "blue" },
    { type = "label", x = 2, y = 1.1, label = "Manually positioned" }
}
```

Container: VBox Example

```
gui.VBox {
    gui.Label { label = "Automatically positioned" },

    -- The Container and its elements are automatically positioned
    -- below the above Label due to them being inside a VBox.
    gui.Container {
        { type = "box", x = 0, y = 0, w = 1, h = 1, color = "red" },
        { type = "box", x = 0.3, y = 0.3, w = 1, h = 1, color = "green" },
        { type = "box", x = 0.6, y = 0.6, w = 1, h = 1, color = "blue" },
        { type = "label", x = 2, y = 1.1, label = "Manually positioned" }
    },
}
```

Container: `noclip` Example

Luanti cuts off elements that stick out of the form unless their `noclip` style is `true`. This example puts a button and a box outside a 2 by 2 area, which is why it needs `noclip`. The `gui.Stack` is only a reference area: it draws nothing, and gives the two containers a known size to be placed against.

```
gui.Stack {
    -- Setting both w and h makes flow skip positioning the elements
    -- inside the stack, so every container keeps its own x and y.
    -- Setting only one of them does not do that.
    w = 2, h = 2,
    padding = 0,

    gui.Container {
        -- A container with no size takes no room, so it only marks a
        -- point.
        w = 0, h = 0,
        x = 0, y = 0,

        -- This moves the container itself: its top-left corner goes to
        -- the middle of the stack (x = 1). It does not move or center
        -- the elements inside it.
        align_h = "center",

        -- Elements inside a container are placed by their top-left corner,
        -- relative to the container. The button is 1 by 1 and ends up at
        -- x = 1, y = -1: directly above the right half of the stack.
        {
            type = "button", padding = 0,
            x = 0, y = -1, w = 1, h = 1,
            label = "Hi",
            style = {
                noclip = true,
                font_size = "*3",
            },
        },
    },

    -- These x-offsets cancel each other out.
    gui.Container {
        w = 0, h = 0,
        x = -1, y = 0,

        -- "start" leaves the container where it is. Alignment applies to
        -- the container, never to the elements inside it.
        align_h = "start",

        -- The container is at x = -1 and the box is 1 to the right of it,
        -- so the box ends up at x = 0, y = -2: above the left edge of the
        -- stack.
        {
            type = "box", x = 1, y = -2, w = 1, h = 1,
            color = "green", style = { noclip = true },
        },
    },
}
```

`noclip` is a style of each element, and the `style` of an element only applies to that element, so set it on every element that sticks out, as both of the elements above do.

Container: Stack Example

```
gui.Stack {
    min_w = 10,
    gui.Button { label = "Hello world!" },

    -- The container is 1 by 1 and is centered on the button.
    gui.Container {
        w = 1, h = 1, padding = 0.2, align_h = "center",
        {
            -- A red square, made with a texture modifier so that this
            -- example does not need any files.
            type = "image",
            x = 0, y = 0, w = 1, h = 1,
            texture_name = "[fill:16x16:red",
        }
    }
}
```

Do not put layout elements (like `gui.VBox`, `gui.HBox`, and `gui.Stack`) inside `gui.Container`, whether you write them with `gui.VBox` or with `type = "vbox"`. A container never lays out its children, it only places them at their `x` and `y`. If you need a box inside a container anyway, this table shows what happens:

Container has `w` and `h`	Box has `w` and `h`	Result
No	No	Works: flow lays out the box. `padding`, `align_h`, and `align_v` on the box are not applied.
No	Yes	Broken: the box's children get no positions.
Yes	No	Broken: flow stops with an error.
Yes	Yes	Broken: the box's children get no positions.

This does not stop you from using a `gui.Container` inside a layouting element, as in the example above. To get the same effect inside a container, put the elements at exact `x` and `y` positions yourself.

4.10 Experimental Features

These features might be broken in the future. Popovers are also experimental, and have their own page.

4.10.1 `no_prepend`

Games and mods can add extra content to the start of every form shown to a player with `player:set_formspec_prepend()`, for example to give all forms the same background or style. This is called a "prepend", and it can be why a form has a background or style you did not expect.

You can set `no_prepend = true` on the "root" element to stop prepends from being applied to a form.

Example:

```
local my_gui = flow.make_gui(function(player, ctx)
    return gui.VBox {
        no_prepend = true,

        gui.Button { label = "Button 1" },
        gui.Button { label = "Button 2" },
    }
end)
```

A prepend is invisible by itself, so to see what `no_prepend` does, this example gives the player a prepend that makes every button red. Tick the checkbox and the red disappears, because the form now ignores the prepend:

```
local gui = flow.widgets
local player = core.get_player_by_name("current_player")

-- Make every button red, like a game's prepend could.
player:set_formspec_prepend("style_type[button;bgcolor=red]")

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Ignore the prepend while the box is ticked.
        no_prepend = ctx.form.skip or false,

        gui.Checkbox { name = "skip", label = "Set no_prepend" },
        gui.Button { label = "Am I red?" },
    }
end)

form:show(player)
```

4.10.2 `bgcolor`

You can set `bgcolor = "#123"`, `fbgcolor = "#123"`, and `bg_fullscreen = true` on the root element to set background colors:

- `bgcolor`: The color of the background of the form itself.
- `fbgcolor`: The color of the fullscreen background, which covers the whole screen behind the form.
- `bg_fullscreen`: Which of the two colors to draw, see the table below.

The value of `bg_fullscreen` decides what is drawn. `true` can look backwards: it does not add the fullscreen background to the form's own one, it switches to the fullscreen one.

`bg_fullscreen`	`bgcolor` (the form)	`fbgcolor` (the screen)
`false` (the default)	drawn	not drawn
`true`	not drawn	drawn
`"both"`	drawn	drawn
`"neither"`	not drawn	not drawn

Flow passes these three values straight to Luanti's `bgcolor[]` formspec element, so Luanti decides what is drawn. This also means that `fbgcolor` is not drawn unless you set `bg_fullscreen` to `true` or `"both"`.

Colors that you leave out are not changed. See Common Aspects for how colors are written.

```
gui.VBox {
    -- Draw a blue form on a dimmed screen
    bgcolor = "#336699",
    fbgcolor = "#00000080",
    bg_fullscreen = "both",
    gui.Label { label = "Hello world!" },
}
```

4.10.3 Window Position

These values allow the position of the displayed form to be moved around and adjust how it is scaled. You will likely need them for most HUDs, which are described in Using a Form as a HUD.

Example:

```
local my_gui = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- window_position is the point on the screen where the form is
        -- placed, and window_anchor is the point on the form that is put
        -- there. For both, 0 is the top/left and 1 is the bottom/right, and
        -- the default is {x = 0.5, y = 0.5} (the center).
        -- You probably want to set `window_position` and `window_anchor` to
        -- the same value.
        -- This puts the form in the bottom-right corner.
        window_position = { x = 1, y = 1 },
        window_anchor = { x = 1, y = 1 },

        -- Minimum space to leave around the form, as a fraction of the
        -- screen size (here 10% on the left/right and 20% at the top/bottom).
        -- If the form is too big, it will be scaled down. 0 lets the form
        -- touch the edge of the screen and negative values let it go off
        -- the edge.
        -- Default for forms: {x = 0.05, y = 0.05} (i.e. 5% of screen size)
        -- HUDs default to a hardcoded pixel size, if you want them to roughly
        -- line up with formspecs then you may explicitly specify this.
        window_padding = { x = 0.1, y = 0.2 },

        gui.Label { label = "Hello world" },
    }
end)
```

4.10.4 Rendering to a Formspec

This API should only be used when necessary and may have breaking changes in the future.

Luanti describes menus with *formspec* strings (see the introduction), which flow normally generates and shows for you. Some APIs in other mods, such as sfinv (see Using a Form as an Inventory), expect formspec strings instead. You can use this API to embed flow forms inside them. To use flow with these mods, you can call `form:render_to_formspec_string(player, ctx, standalone)`.

- A complete formspec starts with a `formspec_version[]` element (the version of the formspec language it uses) and a `size[]` element (the size of the form). By default these aren't included in the returned formspec, and are returned in a third value instead: a table `info` with `w` and `h` (the size of the form, in the units described in Sizes) and `formspec_version` (the version number). Set `standalone` to include them in the returned formspec string. The third return value will not be returned (it is `nil`).
- Returns `formspec, process_event[, info]`, see the two forms below. `info.formspec_version` is always 7, whatever the player's client supports. Pass the same `ctx` again when you redraw so the form keeps its values.
- Call `process_event(fields)` with the `fields` table that Luanti passes to `core.register_on_player_receive_fields` callbacks. It returns `true` if the formspec should be redrawn, where `render_to_formspec_string` should be called and the new `process_event` should be used in the future. It returns `true` when an `on_event` or `on_key_enter` function returned `true`, or when the player changed a value that the build function read from `ctx.form`. This is also the case when `fields.quit` is set (for example when the player clicks a `gui.ButtonExit` whose `on_event` returns `true`), because flow does not

look at `fields.quit` itself. The form is closing then, so callers normally skip the redraw, as the example below does.

The two call forms side by side:

```
-- Embeddable: size and version come back as a third value.
local fs, process_event, info =
    form:render_to_formspec_string(player, ctx)
-- info.w, info.h and info.formspec_version (always 7) are set.

-- Standalone: fs starts with formspec_version[] and size[], and there is
-- no third value.
local fs, process_event =
    form:render_to_formspec_string(player, ctx, true)
```

Here is a complete example. It wraps a flow form in a formspec of its own and shows it with `core.show_formspec`. The label above the form belongs to the outer formspec, and the counter keeps working because `process_event` handles the button click:

```
local gui = flow.widgets

local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = "I am a flow form inside a formspec!" },
        gui.Label { label = "Clicks: " .. (ctx.clicks or 0) },
        gui.Button {
            label = "Click me",
            on_event = function(player, ctx)
                ctx.clicks = (ctx.clicks or 0) + 1
                return true
            end,
        },
    }
end)

local FORMNAME = "mymod:outer"
local contexts = {}
local handlers = {}

local function show(player)
    local name = player:get_player_name()
    contexts[name] = contexts[name] or {}

    -- Pass the same ctx every time so values survive redraws.
    local fs, process_event, info =
        form:render_to_formspec_string(player, contexts[name])
    handlers[name] = process_event

    -- Put the generated elements inside a formspec of our own.
    core.show_formspec(name, FORMNAME, table.concat({
        ("formspec_version[%d]"):format(info.formspec_version),
        ("size[%f,%f]"):format(info.w + 2, info.h + 3),
        "label[1,0.7;This label belongs to the outer formspec]",
        "container[1,1.5]", fs, "container_end[]",
    }))
end

core.register_on_player_receive_fields(function(player, formname, fields)
    if formname ~= FORMNAME then return end

    -- Redraw when flow asks for it.
    local process_event = handlers[player:get_player_name()]
    if process_event and process_event(fields) and not fields.quit then
        show(player)
    end
end)

show(core.get_player_by_name("current_player"))
```

> **Warning:** Do not use this API with node meta formspecs (formspecs that are stored in the metadata of a node), it can and will break!

4.10.5 Embedding a Form Into Another Form

You can embed form objects inside others like this:

```
local gui = flow.widgets

local other_form = flow.make_gui(function(player, ctx)
    return gui.Label { label = "I am embedded" }
end)

local parent_form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = "Hello world" },
        other_form:embed {
            -- A name for the embed, see below.
            name = "my_embed_name",
        },
    }
end)
```

Current versions of flow ignore a `player` parameter in the table passed to `embed`. Versions from before 2025-06-17 need `player = player`, so add it if your mod has to work with them.

Special characters (excluding `-` and `_`) are not allowed in embed names.

The `name` changes a lot about how the embedded form works:

	With `name`	**Without `name`**
`ctx` in the embedded form	Its own table, at `ctx.my_embed_name` in the parent's `ctx`	The same `ctx` as the parent
`ctx.form` in the embedded form	Reads and writes go to the parent's `ctx.form`, with the field names rewritten (the name of the embed is added to the start)	The same `ctx.form` as the parent
Field names, scrollbar names, tooltip names, and style selectors	Rewritten, so they cannot clash with the parent's	Not rewritten, so a name used in both forms is the same input
`on_event`, `on_quit`, and `on_close_popover` functions	Called with the embedded form's own `ctx`	Called with the shared `ctx`
`on_key_enter` functions	Called with the parent's `ctx`, not the embedded form's (a known limitation, see below)	Called with the shared `ctx`

Events and redraws work through the parent. The embedded form's elements are part of the parent's form, so a callback that returns `true` redraws the whole parent form, and `update` and `update_where` called on the embedded form object do nothing, because the form that is open belongs to the parent. Call them on the parent form object instead.

Known limitation: the `on_key_enter` function of a field in a form embedded with a `name` is called with the parent's `ctx`, unlike `on_event`, which gets the embedded form's own `ctx`. In that function `ctx.form.my_field` is `nil`, because the value is stored under the rewritten field name, and anything you store in `ctx` goes into the parent's `ctx`. To reach the embedded form's own `ctx` from there, go through the embed's entry in the parent's `ctx`, for example `ctx.my_embed_name.form.my_field`. Embeds without a `name` are not affected, because they share the parent's `ctx` anyway.

4.10.6 Running Code When a Form Is Closed

`gui.Container`, `gui.HBox`, `gui.VBox`, and `gui.Stack` elements support an `on_quit` callback which gets run when a player closes a form (for example by pressing Escape or clicking an exit button).

Note that this function is not called in some cases, such as when the player leaves without closing the form or when another form/formspec is shown.

This function must not return anything, behaviour may get added to return values in the future.

```
local gui = flow.widgets

-- A second form that pops up once the first one is closed.
local closed_notice = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Label { label = "Form closed!" },
        gui.ButtonExit { label = "OK" },
    }
end)

local my_gui = flow.make_gui(function(player, ctx)
    return gui.VBox {
        on_quit = function(player, ctx)
            core.chat_send_player(player:get_player_name(), "Form closed!")

            -- Defer so the new form is not closed along with this one.
            core.after(0, function()
                closed_notice:show(player)
            end)
        end,

        gui.Label { label = "Press Escape or the button to close me." },
        gui.ButtonExit { label = "Close" },
    }
end)

my_gui:show(core.get_player_by_name("current_player"))
```

If multiple `on_quit` callbacks are specified in different elements, they will all get called.

4.10.7 Handling Enter Keypresses in Fields

Several options deal with the Enter key, and they overlap:

- `on_key_enter` is a function that is called when the player presses Enter in a `gui.Field` or `gui.Pwdfield`.
- `on_event` on the same field is called whenever the client submits the field, which includes Enter but is not limited to it. Use `on_key_enter` to react to Enter.
- `close_on_enter` makes Enter close the form. It is the same as `gui.FieldCloseOnEnter` and only applies to `gui.Field`. Without an `on_key_enter` callback flow logs a warning, because closing is the only thing Enter would do.
- `enter_after_edit` makes Android's "Done" button count as Enter. It is the same as `gui.FieldEnterAfterEdit`.

Notes:

- If you're using this callback, please make sure there's some other way to trigger the enter action (like a button) so that the form works on older flow versions, and in case this API is replaced by a better one in the future.

- If you want recent mobile clients to call this callback when editing text, add `enter_after_edit = true` to the field definition. Without it, pressing "Done" on the on-screen keyboard of an Android device does not count as pressing enter. This is experimental.
- The similarly named `on_event` gets called whenever the client submits the field to the server, which could be at any time, and is not very useful, but is still supported for compatibility (and there may be uses for it, such as sanitising field values). Be careful not to accidentally use the wrong callback.

`gui.Field` and `gui.Pwdfield` support an `on_key_enter` callback that gets called if enter is pressed:

```
local form = flow.make_gui(function(player, ctx)
    return gui.VBox {
        gui.Field {
            label = "Press enter!",
            name = "field",
            on_key_enter = function(player, ctx)
                core.chat_send_player(player:get_player_name(),
                    "Field value: " .. dump(ctx.form.field))

                -- Show the result in the form as well.
                ctx.enters = (ctx.enters or 0) + 1
                ctx.last_value = ctx.form.field
                return true
            end,

            -- Pressing enter does not close the form by default. You can
            -- specify close_on_enter to close it when enter is pressed.
            -- This only applies to gui.Field.
            close_on_enter = ctx.form.close or false,
        },
        gui.Checkbox { name = "close", label = "Close the form on enter" },
        gui.Label {
            label = ("Enter pressed %d times, last value: %s"):format(
                ctx.enters or 0, dump(ctx.last_value)),
        },
    }
end)
```

Type some text and press enter. The label under the field counts the key presses and shows the last text. If you tick the checkbox first, pressing enter closes the form as well.

4.10.8 Getting a Reference to the Current Player and Context

If you're making a custom element, it might be useful to get a reference to `ctx` and `player` so you don't have to pass them in manually.

```
local function HelloWorld(def)
    local ctx, player = flow.get_context()
    return gui.Label {
        label = "Hello, " .. player:get_player_name() .. "!\n" ..
            "some_value=" .. ctx.some_value,
        style = def.style,
    }
end

local form = flow.make_gui(function(player, ctx)
    ctx.some_value = 123
    return HelloWorld { style = { font_size = "*2" } }
end)
```

`flow.get_context()` will error when called outside a build function (as you should not normally do this).

4.10.9 `gui.Flow` Container

`gui.Flow` is a fixed-width container that wraps items around if they don't fit. Unlike other elements, you must specify a width, and some options like `min_w` and `padding` aren't supported.

This is similar to GTK's `FlowBox`, but with a lot fewer features.

The width also has to leave room for the spacing between elements. `spacing` has the same default as for `gui.VBox` (0.2, see also Padding). Flow needs `w` to be at least `1 + 0.2 + 1 + 0.2 + 1` (3.4) to fit three 1-wide elements on one row, and at least 4.6 to fit four, so `3 * (1 + 0.2)` (3.6) is a safe width for three.

```
gui.Flow {
    -- Fits 3 items, including the default spacing
    w = 3 * (1 + 0.2),

    -- Five colored squares: three on the first row, two on the second.
    gui.Box { w = 1, h = 1, color = "#E74C3C" },
    gui.Box { w = 1, h = 1, color = "#E67E22" },
    gui.Box { w = 1, h = 1, color = "#F1C40F" },
    gui.Box { w = 1, h = 1, color = "#2ECC71" },
    gui.Box { w = 1, h = 1, color = "#3498DB" },
}
```

4.11 Popovers (Highly Experimental)

This API will likely have breaking changes in the future.

Flow supports a highly experimental low-level API for defining popovers. Popovers appear over top of all other elements and are anchored to their parent.

This API is intended to be used for creating themed elements like dropdowns, or for a higher level popover API that adds styling and is easier to use.

You add a `popover` attribute to any container element to define a popover. Flow does not manage the lifecycle (opening/closing) of popovers, you must ensure that you do this yourself like in the example.

Two parameters go on the container that owns the popover:

- `popover` : the element to show over the container, or `nil` for no popover.
- `on_close_popover` : a function that flow calls with `(player, ctx)` when the player clicks anywhere outside the popover. Like an `on_event` function, it can return `true` to redraw the form. Flow only adds this click catcher while a popover is shown.

Here is the smallest useful popover: a button that opens a popover with one label, and `on_close_popover` to close it again. The popover is only given to the container while `ctx.show_popover` is set, and is `nil` otherwise (`cond and element or nil` is the usual Lua way to write that):

```
local gui = flow.widgets

local my_gui = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Leave room around the button so the popover stays inside the form.
        padding = 3,

        gui.Stack {
            gui.Button {
                label = "Open popover",
                on_event = function(player, ctx)
                    ctx.show_popover = true
                    return true
                end,
            },

            popover = ctx.show_popover and gui.VBox {
                padding = 0.2,
                bgcolor = "#222e",
                gui.Label { label = "Hi there!" },
            } or nil,

            -- Called when the player clicks outside the popover.
            on_close_popover = function(player, ctx)
                ctx.show_popover = false
                return true
            end,
        },
    }
end)
```

Where the Popover Goes

`side` decides which edge of the container the popover is attached to: `"bottom"` (the default, also used for any other value), `"top"`, `"left"`, or `"right"`. The popover is then lined up along that edge with `align_h` for `"top"` and `"bottom"`, or `align_v` for `"left"` and `"right"` (the other one is ignored). The alignment is relative to the container, not to the form:

Alignment	Result
`"start"`	The popover's left (or top) edge lines up with the container's.
`"end"`	The popover's right (or bottom) edge lines up with the container's.
`"center"`	The popover is centered on the container.
`"fill"`, or unset	The popover is stretched to the width (or height) of the container.

The full example below puts `side` in a dropdown inside the popover, so that you can try every side. `side = ctx.form.side` is `nil` on the first draw, which means `"bottom"`, and the dropdown stores its choice in `ctx.form.side`. Elements inside a popover that are not named keys (such as the label and the dropdown) are its children, in the same table as the named keys.

The popover is drawn from the build function, so it can depend on `ctx` :

```
local gui = flow.widgets

local my_gui = flow.make_gui(function(player, ctx)
    return gui.VBox {
        -- Some padding is added for the purposes of the demo so that the popup
        -- isn't off-screen
        padding = 3,
        gui.Label { label = "Popover demo:" },

        gui.Stack {
            gui.Button {
                label = "Open popover",

                -- When the popover button is clicked, open the popover
                on_event = function(player, ctx)
                    ctx.show_popover = true
                    return true
                end,
            },

            -- The actual element to overlay
            -- "popover" is set to the gui.VBox element if (and only if)
            -- ctx.show_popover is true, otherwise it's nil so no popover is
            -- shown.
            popover = ctx.show_popover and gui.VBox {
                -- Specifying padding and bgcolor is optional, but is probably
                -- a good idea since flow doesn't do any styling on its own.
                padding = 0.2,
                bgcolor = "#222e",

                -- "side" specifies how the popover is positioned relative to
                -- the parent, and can be "bottom" (default), "top", "left", or
                -- "right". It is nil on the first draw, which means "bottom".
                side = ctx.form.side,

                -- align_h and align_v align the popover according to its
                -- parent element. You only need to specify align_h for
                -- side = "top"/"bottom" or align_v for side = "left"/"right",
                -- this example specifies both so that it can demonstrate
                -- switching between different sides.
                align_h = "fill",
                align_v = "center",

                -- Popover contents
                gui.Label { label = "Hi there!" },
                gui.Dropdown {
                    name = "side",
                    items = {
                        "bottom",
                        "top",
                        "left",
                        "right",
                    },
                },
            } or nil,

            -- Clicking outside the popover will call this function, which
```

```
            -- should probably close the popover.
            on_close_popover = function(player, ctx)
                ctx.show_popover = false
                return true
            end,
        },

        gui.Label { label = "After popover button" },
    }
end)
```

Show the form with `my_gui:show(player)` as usual.

`align_h` and `align_v` are among the parameters shared by all elements.

There are some restrictions on popovers:

- They must not extend outside the form, otherwise they'll only be partially visible (unless everything is styled with `noclip = true`). Flow does not attempt to detect this.
- You can only define `popover` on container elements, like `gui.Stack`, `gui.HBox`, and `gui.VBox`.
- Only one popover is shown at a time.
- Players can still use tab to interact with things behind the popover, despite being unable to use their mouse to do so.
- You cannot show popovers inside of other popovers.
- Popovers cannot be on the root element.